

AUDITORIA DE SEGURANÇA

Relatório de Auditoria de Segurança ORÇACLOUD / ÒPURA

01 de setembro de 2026

4

CRÍTICA

8

ALTA

5

MÉDIA

4

BAIXA

1

INFORMATIVA

ESCOPO AUDITADO

Repositório orçacloud-saas (branch main): 24 Edge Functions Deno, ~800 migrations SQL, ~235 serviços e ~340 componentes React, scripts de build/CI e arquivos de configuração. A postura efetiva de RLS, as policies e as ACLs de função foram lidas do banco Postgres REMOTO de produção — não das migrations —, porque o próprio repositório documenta drift no histórico de migrations. Onde havia policy permissiva, a exploração foi confirmada na prática: por requisição HTTP real ao PostgREST com a chave anon pública, e — para os achados críticos — assumindo os papéis anon e authenticated no banco de produção. Toda prova que escreve roda dentro de BEGIN ... RAISE EXCEPTION, que aborta a transação por construção: nenhum dado foi criado, alterado ou removido durante esta auditoria.

NOTA METODOLÓGICA

As cinco categorias do pedido foram mapeadas para a stack detectada abaixo. O projeto não possui ORM nem servidor de aplicação próprio: o isolamento de inquilino é feito por Row Level Security do PostgreSQL, e o papel de “rota de backend” cabe às 24 Edge Functions Deno, que usam a service role e por isso operam fora da RLS. O detalhamento do mapeamento de cada categoria está na seção 1.

1. Stack detectada e mapeamento das categorias

A auditoria começou pela detecção da stack, porque cada categoria do pedido só faz sentido depois de traduzida para o equivalente real deste projeto. O quadro abaixo é o que foi detectado; a tabela seguinte diz, categoria por categoria, o que foi efetivamente procurado e onde.

Camada	Tecnologia detectada
Frontend	React 19 + TypeScript 5.8 (strict), Vite 6, Tailwind v4, Zustand + TanStack Query
Backend	Supabase — PostgreSQL 17 + PostgREST + Auth (GoTrue) + Storage + Realtime
Rotas de servidor	24 Edge Functions em Deno (supabase/functions/*/index.ts)
Camada de dados	Sem ORM. Query builder do supabase-js, chamado por ~235 serviços em services/
Autenticação	Supabase Auth (JWT). Papel no app via organization_members.role
Isolamento de tenant	Row Level Security por organization_id, via is_org_member() / is_org_manager()
Deploy	Vercel (frontend) + Supabase CLI (functions/migrations). Sem Docker/Helm/Terraform

Como cada categoria foi mapeada

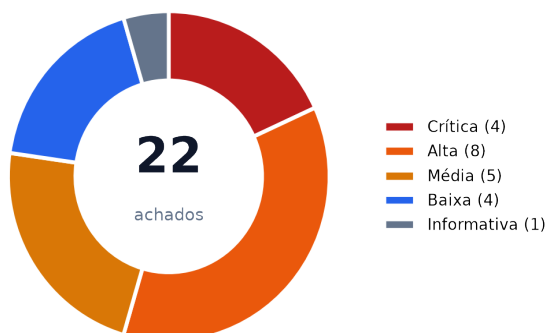
Categoria	Equivalente nesta stack e método aplicado
1. Banco sem tranca	O mecanismo de isolamento deste projeto é RLS por organization_id, apoiada nas funções SECURITY DEFINER is_org_member() e is_org_manager() sobre organization_members. Auditado consultando o banco remoto (pg_class.relrowsecurity, pg_policies, pg_proc.proacl), e não as migrations. Cada policy permissiva encontrada foi testada por requisição HTTP real.
2. Permissão definida no navegador	Não há servidor de aplicação próprio: o equivalente são as Edge Functions, que usam a serviceRoleKey e portanto ignoram a RLS. Cada uma foi lida por inteiro procurando o par “valida que existe um usuário” sem o par “valida QUAL usuário é e o que ele pode fazer”.
3. IDOR	Percorridos os 24 handlers de Edge Function, um a um, e todas as funções SECURITY DEFINER do schema public executáveis pelo papel anon. Critério: todo id que chega pelo corpo, path ou query e é usado num acesso com service_role precisa de checagem explícita de posse, porque nesse caminho a RLS não protege.
4. Chaves expostas	git grep por padrões de segredo em todo o conteúdo versionado, mais inspeção de .env*, .gitignore, supabase/config.toml, migrations, scripts e CI, com atenção a valores de fallback em COALESCE(...) e ?? que viram segredo real quando a variável não é definida.
5. Inputs sem tratamento	No frontend: varredura por dangerouslySetInnerHTML, innerHTML/outerHTML/insertAdjacentHTML, eval e new Function, cruzada com a policy de escrita da tabela que alimenta cada sink. No backend: montagem de HTML de e-mail por interpolação nas Edge Functions.

2. Resumo executivo

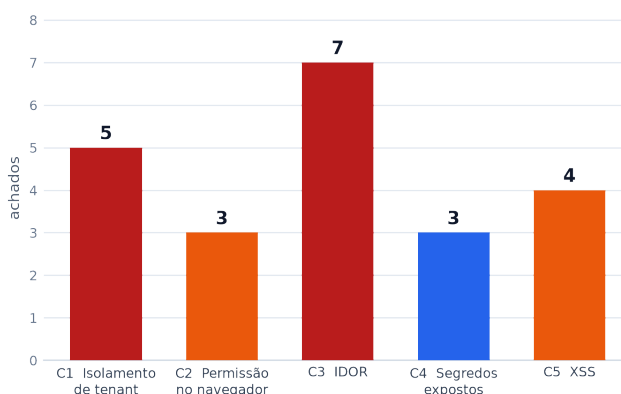
Foram confirmados **22 achados** no código real, distribuídos pelas cinco categorias: **4 críticos**, **8 altos**, **5 médios**, **4 baixos** e **1 informativo**. Os três críticos foram explorados de fato contra o banco de produção, não apenas inferidos da leitura do código: a tabela `invoices` devolveu 829 registros a uma requisição HTTP sem nenhum login; a escalada a `owner` levou um usuário sem vínculo de 0 a 2.214 lançamentos financeiros com um único INSERT; e o papel anônimo emitiu credencial válida de Portal do Cliente e leu os dados cadastrais do titular. As duas provas que escrevem rodaram dentro de transação abortada — nada foi persistido. Foram registrados também **12 pontos fortes** com evidência, que constam da seção 3.

A leitura de conjunto é que o projeto acertou o desenho do isolamento — RLS habilitada em toda a base, funções auxiliares corretas, limpeza documentada de 81 policies permissivas — e errou em três pontos que anulam boa parte desse esforço: uma policy que deixa qualquer usuário se declarar dono de qualquer organização, duas funções que emitem credencial de portal sem pedir autorização, e o hábito de tratar o `organization_id` recebido do cliente como fato dentro das Edge Functions que rodam com `service role`.

ACHADOS POR SEVERIDADE



ACHADOS POR CATEGORIA



Índice dos achados

ID	Sev.	Achado	Local
C1-01	CRÍTICA	Qualquer usuário autenticado se auto-promove a owner de qualquer organização	.../migrations/20260215000009_fix_org_rls_recursive.sql
C1-02	CRÍTICA	Tabela invoices legível por anônimo e por qualquer tenant (829 registros expostos)	.../migrations/20260516100000_fix_invoices_rls_boletos.sql
C3-01	CRÍTICA	RPCs de credencial de portal executáveis por anônimo — três sem guarda nenhuma	.../migrations/20261128000001_client_portal_tokens.sql
C3-02	CRÍTICA	Portal do Colaborador inteiro exposto: anônimo lê folha de pagamento só com o UUID do colaborador	.../labor-portal-ged-download/index.ts + 8 RPCs SECURITY DEFINER
C1-05	ALTA	A perna "OR is_shared" das policies de leitura ignora a organização (127 registros cruzam o tenant)	.../migrations/aplicar_20270914000021_org_not_null_e_policies_is_shared.sql
C2-01	ALTA	sinapi-import: qualquer usuário logado reescreve a base de preços SINAPI de todos os tenants	supabase/functions/sinapi-import/index.ts
C2-02	ALTA	send-bi-report é um relay de e-mail aberto a qualquer usuário autenticado	supabase/functions/send-bi-report/index.ts

ID	Sev.	Achado	Local
C2-03	ALTA	organization_id vem do corpo da requisição e nunca é conferido contra a associação do chamador	supabase/functions/asaas-charge/index.ts
C3-03	ALTA	sign-contract altera assinatura de contrato de qualquer tenant e consulta documento alheio no ZapSign	supabase/functions/sign-contract/index.ts
C3-04	ALTA	asaas-webhook falha aberto: sem a variável de ambiente, aceita qualquer POST	supabase/functions/asaas-webhook/index.ts
C5-01	ALTA	XSS armazenado na Academia: qualquer membro injeta HTML servido a todos os matriculados	components/academy/AcademyLessonPlayer.tsx
C5-02	ALTA	Template de contrato injetado via innerHTML na geração de PDF	components/ContractDetailView.tsx
C1-03	MÉDIA	Policies anon FOR ALL USING(true) sobrevivendo em opura_cno_areas e opura_cno_reductions	pg_policies (banco remoto)
C3-05	MÉDIA	asaas-charge action 'resend' envia boleto de outro tenant para e-mail escolhido pelo atacante	supabase/functions/asaas-charge/index.ts
C3-06	MÉDIA	notify-broker-proposal e notify-opportunity-interest não verificam Authorization nenhuma	.../notify-broker-proposal/index.ts
C3-07	MÉDIA	partner-portal-upload grava em bucket público com contractId arbitrário e content-type do usuário	.../partner-portal-upload/index.ts
C5-03	MÉDIA	E-mails HTML montados por interpolação, com campo vindo de formulário público anônimo	.../notify-opportunity-interest/index.ts
C1-04	BAIXA	payment_types compartilhado entre todos os tenants apesar de ter organization_id	pg_policies (banco remoto)
C4-01	BAIXA	Chave publishable e project ref do Supabase embutidos em script versionado	check_suppliers.js
C4-02	BAIXA	Placeholders de segredo como fallback de COALESCE nos agendamentos pg_cron	.../migrations/20260224000002_setup_billing_cron.sql
C5-04	BAIXA	Preview de template renderiza HTML do próprio autor sem sanitização (self-XSS)	components/ContractTemplateManager.tsx
C4-03	INFORMATIVA	Project ref do Supabase cravado como fallback de Access-Control-Allow-Origin	.../process-billing-ruler/index.ts

3. Pontos fortes (o que está protegido)

Esta seção existe para provar a cobertura da auditoria: são os controles que foram efetivamente verificados e passaram. Cada item traz a evidência que sustenta o veredito.

OK **RLS habilitada em toda a base, sem exceção de negócio**

pg_class.relrowsecurity: a única tabela do schema public com RLS desligada é spatial_ref_sys, de referência do PostGIS. As 15 tabelas com zero policies (backups _bkp_*, contadores, order_chats, commercial_*) ficam inacessíveis por PostgREST – negam por padrão, que é o comportamento seguro.

OK **A limpeza das policies anon de desenvolvimento foi real e verificável**

A migration 20270208000002 removeu 81 policies que davam acesso irrestrito ao papel anon. Testado ao vivo com a chave anon pública: suppliers, clients e internal_transactions retornam Content-Range */0. Só invoices ficou de fora (C1-02) – e a própria migration já registrava a ressalva nas linhas 30-32.

OK **Nenhuma view exposta ao papel anon**

Varredura de pg_class (relkind v/m) com has_table_privilege('anon', ...): restam apenas geography_columns e geometry_columns, ambas do PostGIS. O passivo histórico de views legíveis por anon foi de fato eliminado.

OK **invite-member é o modelo correto de autorização no servidor**

supabase/functions/invite-member/index.ts:41-60 valida o JWT com um cliente anon, consulta organization_members pela organizationId E pelo e-mail do usuário, e exige role em ['admin', 'owner'] antes de qualquer ação privilegiada. É o único ponto do sistema que faz o ciclo completo – e o padrão que C2-01, C2-02 e C2-03 deveriam seguir.

OK **As funções de cron falham fechado**

process-billing-ruler/index.ts:19-25, quality-sla-enforcement/index.ts:26-33 e task-alert-notifier/index.ts:28-31 exigem Authorization exatamente igual a 'Bearer <service_role>' e retornam 401 caso contrário. Sem modo permissivo e sem fallback – o oposto do defeito de C3-04.

OK **academy-portal-media documenta e aplica o padrão seguro de portal**

index.ts:25-35 explicita duas decisões: nunca aceitar storagePath do cliente (o caminho é derivado no servidor a partir de lessonId/materialId/certificateId) e tirar o recorte do token em portal_tokens, não de um employeeId cru “porque seria enumerável”. Confere matrícula ativa antes de assinar e registra toda abertura em academy_access_logs.

OK **Os três portais de download validam vínculo antes de assinar a URL**

supplier-portal-download/index.ts:38-57 confere token e casa file_path com supplier_id; portal-ged-download/index.ts:38-64 confere token e exige que o path seja a versão ativa de documento compartilhado com aquele cliente; partner-portal-download/index.ts:61-71 delega à RPC partner_portal_can_download, que cobre compartilhamento avulso e por pasta. Nos três, o service_role só entra depois da checagem.

OK **supplier-portal-upload trata o upload com cuidado incomum**

index.ts:39-101: limite de 5MB antes de ler o arquivo, validação do token, conferência de que orderId pertence àquele supplier_id (linhas 57-65), sanitização do nome do arquivo por allowlist de caracteres com truncamento, caminho prefixado pelo supplier_id do token e remoção do objeto no storage se a gravação no banco falhar.

OK **eval e new Function foram deliberadamente eliminados do cálculo de folha**

services/payrollEngine.ts:8 documenta um parser aritmético próprio que “substitui new Function() / eval()”. A varredura por eval(e new Function(em components/, services/, utils/, lib/ e hooks/ não encontrou nenhuma outra ocorrência.

OK **Nenhum segredo real versionado e .gitignore correto**

git grep por eyJhbGciOi, service_role, sb_secret_, sk-ant-, \$aact_ e padrões de api_key em todo o conteúdo versionado não retornou nenhum segredo real – só nomes de variável e instruções de documentação. O .gitignore cobre .env, .env.local e .env*.local, e git ls-files confirma que apenas .env.example está versionado, com placeholders.

OK

As funções notify-* filtram por organizationId nas consultas

notify-opportunity-interest/index.ts:58-76 cruza opportunityId com organizationId e interestId com opportunityId; notify-broker-proposal/index.ts:43-55 cruza proposalId com organizationId. Impede montar notificação com dado de outro tenant — o defeito ali é a ausência de autenticação (C3-06), não o escopo da consulta.

OK

As regras obrigatórias do projeto têm verificação mecânica no CI

scripts/check-*.sh e __tests__/orgContextGuard.test.ts, este último rodando no CI a cada push/PR como catraca com BASELINE que só pode diminuir. É a estrutura certa para transformar cada achado deste relatório em regressão impossível.

4. Pontos fracos (os riscos centrais)

Não é a lista de achados, é a leitura do que os produz. Corrigir o padrão abaixo impede a próxima ocorrência; corrigir só o achado, não.

A tabela que define quem é membro pode ser escrita por qualquer um

C1-01 não é um furo a mais na RLS: é o furo que anula a RLS inteira. Toda policy org-scoped do sistema pergunta a organization_members quem você é, e qualquer autenticado pode responder por si mesmo, escolhendo o papel. Enquanto essa policy existir, os controles corretos das outras tabelas não valem nada, e nenhuma outra correção deste relatório muda o resultado final.

Edge Function com service_role vira caminho paralelo que ignora a RLS

O projeto investiu pesado em RLS, mas 11 das 24 funções criam um cliente com serviceRoleKey. Nesse caminho a RLS deixa de existir e a autorização passa a ser responsabilidade manual do handler — e o padrão dominante é confundir autenticação com autorização: confirma-se que existe um usuário, nunca que aquele usuário pode agir sobre aquele objeto daquela organização (C2-01, C2-02, C2-03, C3-03). O organization_id chega pelo corpo da requisição e é tratado como fato.

Regras de compartilhamento escritas pela metade

C1-05 é o mesmo tipo de erro que C1-01, num lugar diferente: a policy diz que o registro é compartilhado, mas não diz com quem, e “OR is_shared” avaliado sozinho é sempre verdadeiro. O nome da policy de suppliers — “Users can view suppliers of their organization” — descreve uma regra que a expressão não implementa, e é justamente esse descompasso entre o nome e a expressão que faz o defeito passar despercebido em revisão. Vale como alerta geral: toda perna de OR numa policy precisa ser lida como “isto sozinho basta para liberar a linha?”.

Credencial de portal emitida sem autorização, por default de GRANT do PostgreSQL

C3-01 nasce de um detalhe de plataforma: toda função nova recebe EXECUTE para PUBLIC, e as migrations concedem a authenticated sem nunca revogar de PUBLIC. O resultado é que duas funções que EMITEM credencial de acesso ficaram abertas ao papel anon. É defeito sistêmico de processo, não esquecimento pontual — o mesmo padrão vale para toda SECURITY DEFINER criada sem o REVOKE.

Identidade de portal ora é token, ora é um id cru enumerável

Há dois desenhos convivendo. O bom usa token opaco com validade e vínculo (academy-portal-media, os três downloads). O ruim aceita o identificador do titular direto no corpo — labor-portal-ged-download com employeeId (C3-02). O código do próprio projeto já diagnosticou a diferença por escrito, e a versão ruim continua em produção.

Nenhuma sanitização de HTML em todo o projeto

Não existe DOMPurify nem helper próprio de escape. São quatro sinks no frontend e duas montagens de e-mail no backend, todos alimentados por tabelas cuja policy de escrita é is_org_member — isto é, o membro de papel mais baixo consegue injetar HTML que será renderizado na sessão de um administrador (C5-01, C5-02). Falta a peça, não a aplicação dela.

Guardas que falham abertas e placeholders que viram segredo

C3-04 só protege se a variável existir ("if (webhookToken && ...)"), e C4-02 substitui segredo ausente por um literal. Nos dois casos o modo inseguro é o comportamento default, e silencioso. Falta validação de inicialização que recuse subir sem os segredos obrigatórios.

5. Achados detalhados por categoria

Cada achado traz arquivo e linha exatos, o trecho de código, por que é explorável, o impacto, a correção sugerida e como o achado foi verificado. Onde a exploração depende de uma condição de ambiente, ela vem destacada.

C1 Banco sem tranca (isolamento de tenant)

CRÍTICA

C1-01 Qualquer usuário autenticado se auto-promove a owner de qualquer organização

supabase/migrations/20260215000009_fix_org_rls_recursive.sql linha(s) 65-67

```
create policy "Authenticated users can create memberships"
on organization_members for insert to authenticated
with check (true);
```

POR QUE É EXPLORÁVEL organization_members é a tabela que define quem pertence a que organização e com que papel. Toda a RLS do sistema depende dela: is_org_member(org) e is_org_manager(org) são SECURITY DEFINER e resolvem a permissão consultando exatamente essa tabela, casando por auth.uid() ou pelo e-mail do JWT. A policy de INSERT tem WITH CHECK (true) — não restringe organization_id, não restringe email e não restringe role. Qualquer conta autenticada (inclusive uma recém-criada por self-signup, que está habilitado em supabase/config.toml) executa um único INSERT informando o próprio e-mail, o organization_id alvo e role='owner'. A partir daí is_org_member e is_org_manager retornam TRUE para aquela organização, e todas as policies org-scoped do sistema — financeiro, folha, contratos, documentos — passam a liberar leitura e escrita. Quebra total do multi-tenant e escalada de privilégio numa única requisição. COMPROVADO EM PRODUÇÃO, em transação abortada. Assumindo o papel authenticated com um e-mail sem nenhum vínculo, o INSERT com role='owner' foi ACEITO pela RLS, e as medições antes/depois na mesma transação foram: vínculos do ator 0 → 1; is_org_member FALSE → TRUE; is_org_manager FALSE → TRUE; linhas visíveis em organizations 0 → 1; em internal_transactions 0 → 2214. Ou seja: de nenhum acesso a 2.214 lançamentos financeiros e direitos de proprietário, com uma instrução. Nada foi persistido — o bloco termina em RAISE EXCEPTION, que aborta a transação por construção.

IMPACTO Tomada de controle completa de qualquer tenant do SaaS por qualquer usuário cadastrado que conheça o UUID da organização alvo.

CONDIÇÃO DE EXPLORABILIDADE O atacante precisa conhecer o organization_id (UUID) do alvo: a RLS de organizations impede listar as organizações, e a primeira execução da prova, sem o UUID, inseriu organization_id NULL e não escalou nada. Isso reduz a superfície, mas não é um controle de segurança — o UUID não é segredo: ele viaja no link de convite (invite-member monta redirectTo como /?org=<uuid>), aparece na URL da aplicação e é conhecido por qualquer pessoa que já tenha sido membro. O cenário realista é o ex-funcionário removido que anotou o UUID e se readiciona como owner.

CORREÇÃO SUGERIDA Trocar o WITH CHECK (true) por uma condição que só permita a linha que o fluxo legítimo precisa, e mover a criação de membro para o servidor. Concretamente: (a) restringir a policy a is_org_manager(organization_id) OR is_superuser(); (b) para o auto-vínculo do primeiro dono ao criar a organização, usar uma RPC SECURITY DEFINER dedicada que só aceite organização sem nenhum membro; (c) o convite de terceiros já tem caminho correto e checado — a Edge Function invite-member.

COMO FOI VERIFICADO Definição lida em pg_policies no banco remoto; cadeia confirmada em pg_get_functiondef de is_org_member e is_org_manager; e exploração executada de ponta a ponta contra o banco de produção dentro de BEGIN ... RAISE EXCEPTION (rollback garantido, zero resíduo).

CRÍTICA**C1-02 Tabela invoices legível por anônimo e por qualquer tenant (829 registros expostos)**

supabase/migrations/20260516100000_fix_invoices_rls_boletos.sql linha(s) 14-24

```
-- Política única: membros autenticados de qualquer organização têm acesso total
CREATE POLICY "invoices_authenticated_all" ON public.invoices
  FOR ALL TO authenticated
  USING (true)
  WITH CHECK (true);
```

POR QUE É EXPLORÁVEL Estado atual no banco remoto: três policies em invoices — “Suppliers can view their own invoices” (SELECT, papel anon, USING true), “Suppliers can insert their own invoices” (INSERT, papel anon, WITH CHECK true) e “invoices_authenticated_all” (ALL, authenticated, USING/CHECK true). Nenhuma tem recorte: o nome fala em “their own”, mas a expressão é literalmente true. A tabela sequer possui coluna organization_id — o vínculo com o tenant é indireto, por supplier_id. A chave anon fica publicada no bundle JavaScript por construção, então a policy anon equivale a acesso público. Verificado ao vivo: GET /rest/v1/invoices com a chave anon do .env, sem qualquer login, retornou HTTP 206 e Content-Range 0-828/829 — 829 notas fiscais de fornecedor de todos os tenants, com nome de arquivo, fornecedor e caminho no storage. O INSERT anon com CHECK true também permite injetar notas forjadas.

IMPACTO Vazamento público de 829 notas fiscais (fornecedor, valor, vencimento, centro de custo, caminho do arquivo) e inserção de notas falsas por anônimo.

CORREÇÃO SUGERIDA Remover as duas policies anon e substituir invoices_authenticated_all por policies com recorte real. Como a tabela não tem organization_id, o caminho mais seguro é (a) acrescentar a coluna com backfill a partir de suppliers/purchase_orders, ou (b) enquanto isso, escrever a policy via EXISTS sobre suppliers com is_org_member(suppliers.organization_id). O Portal do Fornecedor não precisa da policy anon: ele já passa pelas Edge Functions supplier-portal-download e supplier-portal-upload, que usam service_role após validar o token.

COMO FOI VERIFICADO Confirmado por requisição HTTP real (HTTP 206, Content-Range 0-828/829) com a chave anon pública.

ALTA**C1-05 A perna “OR is_shared” das policies de leitura ignora a organização (127 registros cruzam o tenant)**

supabase/migrations/aplicar_20270914000021_org_not_null_e_policies_is_shared.sql linha(s) 67-69

```
CREATE POLICY "Allow authenticated users to read clients"
ON public.clients FOR SELECT TO authenticated
USING (public.is_org_member(organization_id) OR is_shared);
```

POR QUE É EXPLORÁVEL A intenção do sinalizador is_shared é permitir que um cadastro seja reaproveitado entre as organizações de um mesmo grupo. Mas a expressão escrita não tem a segunda metade da regra: “OR is_shared” é verdadeiro sozinho, sem nenhuma condição sobre quem está lendo. Não é “compartilhado com as organizações do grupo” — é compartilhado com TODOS os usuários autenticados do SaaS inteiro, inclusive clientes concorrentes que nunca tiveram relação com aquele tenant. O mesmo padrão está em três policies: clients, suppliers (“Users can view suppliers of their organization” — o nome contradiz a expressão) e partner_workspaces. **COMPROVADO EM PRODUÇÃO** (somente leitura). Assumindo o papel authenticated com um e-mail sem nenhuma linha em organization_members, a contagem de linhas visíveis foi: clients 7, suppliers 119, partner_workspaces 1 — enquanto organizations retornou 0, o que prova que a identidade usada realmente não tinha vínculo nenhum. São 119 de 244 fornecedores (49% da base) e 7 de 46 clientes expostos a qualquer conta do SaaS. Os registros de clients carregam PII: nome, CPF/CNPJ, endereço e telefone.

IMPACTO	Vazamento cross-tenant de 127 cadastros — incluindo CPF/CNPJ, endereço e telefone de pessoas físicas — para qualquer usuário autenticado de qualquer organização do SaaS.
CONDIÇÃO DE EXPLORABILIDADE	O impacto REAL hoje é baixo e o defeito é uma bomba-relógio, não um vazamento em curso: as quatro organizações existentes no banco pertencem todas ao mesmo cliente (Alpa Construtora, ALPA Empreendimentos, a SPE Garden Cambuhy e uma organização pessoal), então os 127 registros circulam hoje apenas dentro do grupo que os possui. O defeito vira vazamento real no instante em que o segundo cliente entrar no SaaS — e é por isso que a correção é pré-requisito de onboarding, não item de manutenção. Registrado assim para não superdimensionar o presente nem subdimensionar o risco.
CORREÇÃO SUGERIDA	Completar a regra: o compartilhamento precisa dizer COM QUEM. Substituir “OR is_shared” por uma condição que exija que o leitor pertença ao mesmo grupo/holding do dono do registro — por exemplo “OR (is_shared AND is_org_member(<org do grupo>))”, ou uma tabela explícita de compartilhamento (organization_id dono, organization_id destino). Aplicar às três policies. Enquanto a regra correta não existir, o caminho seguro é remover a perna is_shared.
COMO FOI VERIFICADO	pg_policies (busca por qual LIKE '%is_shared%') no banco remoto, contagem de linhas is_shared por tabela, e leitura executada como papel authenticated sem vínculo, em transação abortada.

MÉDIA**C1-03 Policies anon FOR ALL USING(true) sobrevivendo em opura_cno_areas e opura_cno_reductions**

pg_policies (banco remoto) linha(s) Allow anon all on opura_cno_areas / opura_cno_reductions

tablename	polycyname	cmd	roles	using
opura_cno_areas	Allow anon all on opura_cno_areas	ALL	{anon}	true
opura_cno_reductions	Allow anon all on opura_cno_reductions	ALL	{anon}	true

POR QUE É EXPLORÁVEL São duas das policies “Regra 8 / Dev” que a migration 20270208000002 se propôs a eliminar (81 removidas) e que escaparam do rollout. FOR ALL com USING(true) para o papel anon significa leitura, escrita e exclusão por qualquer portador da chave pública. A exploração hoje é limitada porque as duas tabelas estão vazias (verificado por PostgREST: Content-Range */0), mas nada impede a escrita — o anônimo pode popular a tabela — e a exposição passa a ser total no dia em que o módulo CNO receber dados de produção. É um buraco latente, não um buraco fechado.

IMPACTO	Leitura e escrita anônima sobre dados de CNO/previdência assim que o módulo entrar em uso.
CORREÇÃO SUGERIDA	DROP das duas policies, no mesmo padrão da migration 20270208000002, deixando só as policies org-scoped de authenticated.
COMO FOI VERIFICADO	pg_policies no banco remoto; contagem de linhas via PostgREST com chave anon (0 linhas hoje).

BAIXA**C1-04 payment_types compartilhado entre todos os tenants apesar de ter organization_id**

pg_policies (banco remoto) linha(s) Allow authenticated users to manage / to read payment types

payment_types	Allow authenticated users to manage payment types	ALL	{authenticated}	true
payment_types	Allow authenticated users to read payment types	SELECT	{authenticated}	true

POR QUE É EXPLORÁVEL A tabela tem coluna de tenant, mas as duas policies usam USING(true) para authenticated. Foi a única tabela com coluna de tenant nessa condição em toda a varredura cruzada entre pg_policies e information_schema.columns. Qualquer usuário logado lê, altera e apaga os tipos de pagamento de qualquer organização. O dado em si é catálogo (baixa sensibilidade), mas a escrita permite sabotagem do cadastro alheio e os nomes podem revelar estrutura financeira de outro tenant.

IMPACTO	Leitura e alteração cruzada de catálogo financeiro entre tenants.
CORREÇÃO SUGERIDA	Reescrever as duas policies com <code>is_org_member(organization_id)</code> , preservando leitura de linhas globais (<code>organization_id IS NULL</code>) se o catálogo tiver seeds do sistema.
COMO FOI VERIFICADO	<code>pg_policies</code> cruzado com <code>information_schema.columns</code> (presença de <code>organization_id</code>).

C2 Permissão definida no navegador

ALTA

C2-01 sinapi-import: qualquer usuário logado reescreve a base de preços SINAPI de todos os tenants

`supabase/functions/sinapi-import/index.ts` linha(s) 47-89

```
const { data: { user }, error: authError } = await userClient.auth.getUser();
if (authError || !user) return json({ error: 'Unauthorized' }, 401);

const adminClient = createClient(supabaseUrl, serviceKey, { ... });
// ... nenhuma checagem de papel entre as duas coisas ...
const { error } = await adminClient
  .from('sinapi_items')
  .upsert(items, { onConflict: 'code,reference_date', ignoreDuplicates: false });
```

POR QUE É EXPLORÁVEL A função verifica apenas que existe um usuário válido — nunca qual o papel dele nem a que organização pertence — e depois grava com `adminClient` (`service_role`), que ignora a RLS. A RLS de `sinapi_items` foi escrita justamente para impedir isso: leitura para `authenticated/anon`, escrita somente para `service_role`. A Edge Function contorna essa proteção em nome de quem chamar. `sinapi_items` e `sinapi_references` são dados GLOBAIS, sem `organization_id`: são a tabela de preços usada por todos os orçamentos de todos os tenants. Do lado do frontend não há sequer gate de papel — `SinapiImportModal` é aberto por `DatabaseExplorer.tsx:1621` sem nenhuma checagem de `isAdmin`. O `upsert` é por (`code`, `reference_date`), então o atacante sobrescreve preços existentes, não apenas acrescenta.

IMPACTO	Corrupção da base de preços que alimenta os orçamentos de todos os clientes do SaaS; orçamentos passam a sair com valores manipulados.
CORREÇÃO SUGERIDA	Exigir papel privilegiado no servidor antes do <code>upsert</code> : consultar <code>organization_members</code> pelo e-mail do usuário validado e aceitar somente <code>owner/admin</code> (ou um papel de <code>superadmin</code> dedicado, já que o dado é global) — exatamente o padrão que <code>invite-member/index.ts:51-60</code> já usa. Adicionar também o gate de papel na UI, para coerência.
COMO FOI VERIFICADO	Código lido integralmente; RLS de <code>sinapi_items/sinapi_references</code> confirmada em <code>pg_policies</code> (escrita só <code>service_role</code>).

ALTA

C2-02 send-bi-report é um relay de e-mail aberto a qualquer usuário autenticado

`supabase/functions/send-bi-report/index.ts` linha(s) 37-71

```
const { data: { user }, error: authError } = await userClient.auth.getUser();
if (authError || !user) return json({ error: 'Token inválido' }, 401);

const { recipients, subject, htmlBody, scheduleId, organizationId } = await req.json();
if (!recipients?.length) return json({ error: 'Nenhum destinatário informado.' }, 400);

const sendRes = await fetch('https://api.resend.com/emails', {
  body: JSON.stringify({ from, to: recipients, subject, html: htmlBody }),
});
```

POR QUE É EXPLORÁVEL	Depois de confirmar que existe um usuário, a função repassa para a Resend, sem nenhuma validação, três campos inteiramente controlados pelo chamador: a lista de destinatários, o assunto e o corpo HTML. O campo organizationId é recebido e nunca usado em consulta alguma. O remetente é o domínio verificado da empresa (relatorios@opura.com.br, linha 57). Qualquer usuário cadastrado — inclusive alguém que acabou de se cadastrar, já que o self-signup está habilitado — dispara e-mail com HTML arbitrário para qualquer endereço, assinado pelo domínio da empresa. É o vetor clássico de phishing com reputação emprestada, e queima a reputação do domínio no envio em massa. O scheduleId também é usado sem checagem de posse: a linha 85 atualiza bi_report_schedules por id, com service_role.
IMPACTO	Phishing e spam saindo do domínio corporativo verificado; atualização de agendamentos de relatório de outros tenants.
CORREÇÃO SUGERIDA	Validar que o usuário é membro de organizationId; derivar os destinatários no servidor a partir de bi_report_schedules daquela organização em vez de aceitar a lista do body; montar o HTML no servidor a partir dos dados do relatório; e filtrar scheduleId por organization_id.
COMO FOI VERIFICADO	Código lido integralmente; nenhuma consulta a organization_members no arquivo.

ALTA

C2-03 organization_id vem do corpo da requisição e nunca é conferido contra a associação do chamador

supabase/functions/asaas-charge/index.ts linha(s) 47-73 (e asaas-payment/index.ts:70-89; sign-contract/index.ts:36-62)

```
const { data: { user }, error: authError } = await userClient.auth.getUser();
if (authError || !user) return json({ error: 'Token inválido' }, 401);

const admin = createClient(supabaseUrl, serviceRoleKey, { ... }); // ignora RLS
const { organization_id, transaction_id } = body;
if (!organization_id) return json({ error: 'organization_id é obrigatório.' }, 400);
// organization_id é usado como filtro, nunca validado contra o usuário
```

POR QUE É EXPLORÁVEL	As três funções seguem o mesmo padrão: validam que existe um usuário, criam um cliente com service_role (que ignora a RLS) e passam a usar o organization_id vindo do corpo como se fosse confiável. Ele aparece em .eq(), o que dá a impressão de escopo — mas o escopo é o que o atacante escolheu, não o que ele tem direito. Um usuário do tenant A que conheça (ou enumere) um transaction_id do tenant B emite cobrança real no Asaas contra o recebível alheio, cancela cobranças do tenant B (action 'cancel', linhas 125-161) e reverte o status do recebível. Em asaas-payment o mesmo vale para boleto_id e supplier_payment_id, com efeito de dinheiro saindo. Como a RLS está fora do caminho, ela não serve de rede de segurança.
IMPACTO	Operações financeiras reais (emissão, cancelamento, pagamento de boleto) executadas sobre dados de outro tenant.
CORREÇÃO SUGERIDA	Nas três funções, logo após getUser(), consultar organization_members filtrando por organization_id e pelo e-mail/uid do usuário, devolvendo 403 se não houver linha — o mesmo bloco que invite-member/index.ts:51-60 já implementa. Extrair para um módulo compartilhado em supabase/functions/_shared/ e reusar em toda função que receba organization_id.
COMO FOI VERIFICADO	Três arquivos lidos integralmente; nenhuma consulta a organization_members em nenhum deles.

C3 IDOR (objeto acessado por id, sem posse)

CRÍTICA

C3-01 RPCs de credencial de portal executáveis por anônimo — três sem guarda nenhuma

supabase/migrations/20261128000001_client_portal_tokens.sql linha(s) 69-89 (e 20261224000001_broker_portal_tokens.sql:42-66)

```
CREATE OR REPLACE FUNCTION public.client_portal_generate_token(p_client_id uuid, p_org_id uuid)
  RETURNS text LANGUAGE plpgsql SECURITY DEFINER AS $function$
DECLARE v_token TEXT;
BEGIN
  v_token := gen_random_uuid()::text;
  INSERT INTO public.client_portal_tokens (org_id, client_id, token)
  VALUES (p_org_id, p_client_id, v_token)
  ON CONFLICT (client_id) DO UPDATE SET token = v_token, ...;
  RETURN v_token;
END; $function$

GRANT EXECUTE ON FUNCTION public.client_portal_generate_token(UUID, UUID) TO authenticated;
-- nunca há REVOKE EXECUTE ... FROM PUBLIC
```

POR QUE É EXPLORÁVEL São OITO as RPCs de credencial de portal com a ACL aberta ao papel anon — os seis emissores (client, broker, investor, partner, supplier e o do colaborador) mais dois revogadores (partner e supplier), que permitiriam a um anônimo REVOGAR o acesso de um parceiro legítimo. ■■

CORREÇÃO DE ESCOPO (2026-09-02): a primeira versão deste achado afirmava que as oito emitiam token 'sem verificar quem chama'. A leitura de pg_get_functiondef das oito, feita ao escrever a correção, mostrou que isso vale para apenas TRÊS: client_portal_generate_token, broker_portal_generate_token e portal_generate_token (colaborador). As outras cinco já chamavam <portal>_portal_can_manage_tokens(p_org_id), que exige organization_members.role IN ('owner','admin') e confere que o titular pertence à organização. Como anon não tem auth.uid() nem auth.jwt(), essa guarda já as tornava inalcançáveis anonimamente, apesar da ACL aberta. O vetor anônimo real existia em três, não em oito — e a prova de conceito abaixo explorou justamente uma delas. O REVOKE segue necessário nas oito: a ACL aberta é defeito por si só, e é o que separava as cinco corretas de uma linha de código. As três desprotegidas eram SECURITY DEFINER (rodam como owner, ignorando RLS), recebiam o id do titular como parâmetro e emitiam um token de portal válido por 90 dias — sem verificar quem chama, sem conferir que o titular pertence a p_org_id e sem exigir sequer sessão. As migrations fazem GRANT EXECUTE TO authenticated, mas nunca fazem REVOKE EXECUTE FROM PUBLIC; como o PostgreSQL concede EXECUTE a PUBLIC por padrão, a ACL efetiva no banco remoto é {=X/postgres, postgres=X, anon=X, authenticated=X, service_role=X} — o “=X” inicial é o PUBLIC, e anon aparece explicitamente. O papel anon executa. A cadeia completa é: chamar client_portal_generate_token, receber um token válido, chamar client_portal_get_data(token) (também SECURITY DEFINER e anon) para ler contratos, parcelas e avisos daquele cliente, e chamar a Edge Function portal-ged-download com o mesmo token para baixar os documentos do GED. O ON CONFLICT DO UPDATE ainda substitui o token legítimo, derrubando o acesso do cliente real — negação de serviço como efeito colateral. COMPROVADO EM PRODUÇÃO, em transação abortada. Assumindo o papel anon (exatamente o que a chave publicada no bundle concede) e sem login nenhum: a leitura direta da tabela clients retornou 0 linhas — a RLS funciona —, mas client_portal_generate_token(<client_id>, <org_id>) executou e devolveu um token UUID válido; em seguida client_portal_get_data(<token>) devolveu o payload do portal com "valid": true e os dados cadastrais do cliente, incluindo nome completo e CPF (redigido neste relatório). Ou seja, a RPC de emissão de credencial contorna por completo a RLS que protege a tabela. Nada foi persistido: o bloco termina em RAISE EXCEPTION, então o token gerado foi descartado e o token legítimo do cliente permanece intacto.

IMPACTO	Sequestro anônimo dos Portais do Cliente, do Corretor e do Colaborador (as três sem guarda). Nas outras cinco, a ACL aberta era defeito latente — a guarda interna as segurava.
CORREÇÃO SUGERIDA	Duas correções, ambas necessárias. (1) REVOKE EXECUTE ON FUNCTION ... FROM PUBLIC, anon nas oito funções, e varrer as demais SECURITY DEFINER pelo mesmo defeito de ACL. (2) Adicionar autorização dentro da função: exigir is_org_manager(p_org_id) e validar que o id do titular pertence a p_org_id — emitir credencial de acesso é operação administrativa e não deve depender só do GRANT.
COMO FOI VERIFICADO	pg_get_functiondef e pg_proc.proacl lidos no banco remoto (ACL efetiva confirma anon=X em ambas); e cadeia emissão → leitura executada como papel anon contra o banco de produção, dentro de BEGIN ... RAISE EXCEPTION (rollback garantido, token descartado).

CRÍTICA**C3-02 Portal do Colaborador inteiro exposto: anônimo lê folha de pagamento só com o UUID do colaborador**

supabase/functions/labor-portal-ged-download/index.ts + 8 RPCs SECURITY DEFINER linha(s) 32-65 (e components/LaborPortal.tsx:78-124)

```
// A ponta visível – a Edge Function:
const { employeeId, storagePath } = await req.json();
const { data: emp } = await admin.from('employees').select('id').eq('id', employeeId).maybeSingle();
if (empError || !emp) return json({ error: 'Colaborador não encontrado' }, 403);
// única checagem: o colaborador EXISTE. Sem token, sessão ou prova de identidade.

// A causa real – a camada de RPC, toda executável por anon e por employee_id cru:
// portal_employee_summary(p_employee_id) portal_get_payroll_runs(p_employee_id)
// portal_get_documents(p_employee_id) portal_get_ged_documents(p_employee_id)
// portal_get_absences(p_employee_id) portal_get_time_entries(p_employee_id)
// portal_get_trainings(p_employee_id) is_employee_shared_with_user(p_employee_id)
```

POR QUE É EXPLORÁVEL A Edge Function não valida token nem sessão: o único controle é que o employeeId informado exista na tabela employees. Mas ela é apenas a ponta visível. A varredura de pg_proc mostrou que o Portal do Colaborador INTEIRO funciona assim: as sete funções de leitura (portal_employee_summary, portal_get_payroll_runs, portal_get_documents, portal_get_ged_documents, portal_get_absences, portal_get_time_entries, portal_get_trainings) são SECURITY DEFINER, recebem p_employee_id cru e são executáveis pelo papel anon. components/LaborPortal.tsx:78-124 as chama exatamente assim, com o employeeId cru. O próprio comentário da Edge Function admite o desenho (“a sessão do portal é anon/employeeId, sem token assinado nem login Supabase”), e a função irmã academy-portal-media documenta explicitamente, nas linhas 32-33, por que está errado: “O recorte vem do TOKEN (portal_tokens), não de um employeeId cru passado pelo cliente. Passar employeeId seria enumerável.” Existe portal_tokens com employee_id, e existe até o padrão pronto do outro lado — fn_portal_get_ged_documents(p_token), usada pelo Portal do Cliente. A primitiva certa está escrita e não foi usada aqui. COMPROVADO EM PRODUÇÃO (somente leitura, papel anon, sem login). O contraste é o que torna o achado inequívoco: anon NÃO tem sequer GRANT SELECT na tabela employees — a consulta direta falha com 42501 e a mensagem do Postgres chega a sugerir o GRANT que falta. Ainda assim, portal_employee_summary(<uuid>) devolveu o cadastro do colaborador (nome, cargo, status) e portal_get_payroll_runs(<uuid>) devolveu as FOLHAS DE PAGAMENTO, com competência e valores. As RPCs SECURITY DEFINER contornam por completo a proteção da tabela.

IMPACTO	Leitura anônima de folha de pagamento, ponto, ausências, treinamentos e documentos de RH de qualquer colaborador de qualquer tenant, bastando o UUID — que também pode ser mintado por anon via portal_generate_token (C3-01).
----------------	--

CORREÇÃO SUGERIDA Criar as variantes `fn_portal_*(p_token text)` das sete funções, derivando o `employee_id` de `portal_tokens` (validando `is_active` e `expires_at`) — mesmo desenho de `fn_portal_get_ged_documents(p_token)`, que já existe. Depois `REVOKE EXECUTE ... FROM PUBLIC, anon` nas sete antigas. Migrar `components/LaborPortal.tsx` para o token e aplicar o mesmo na Edge Function `labor-portal-ged-download`. Nenhum identificador de titular deve chegar cru pelo corpo da requisição.

COMO FOI VERIFICADO `pg_proc` (`prosecdef + has_function_privilege` para `anon`) no banco remoto; chamadas executadas como papel `anon` contra produção, em transação abortada, devolvendo cadastro e folha de pagamento; e `components/LaborPortal.tsx:78-124` confirmado passando `employeeId` cru.

ALTA

C3-03 sign-contract altera assinatura de contrato de qualquer tenant e consulta documento alheio no ZapSign

`supabase/functions/sign-contract/index.ts` linha(s) 107-163

```
if (target === 'document_version') {
  await adminClient.from('contract_document_versions').update({
    signature_token: zapDoc.token, signature_status: 'SENT', signature_url: signUrl,
  }).eq('id', documentVersionId); // id veio do body, sem checagem de posse
...
if (action === 'status') {
  const { signatureToken } = body;
  const zapResp = await fetch(`${ZAPSIGN_API}/docs/${signatureToken}/`, { ... });
  return json({ status: zapDoc.status, signers: zapDoc.signers, signed_file: zapDoc.signed_file });
}
```

POR QUE É EXPLORÁVEL Depois de validar que existe um usuário, a função escreve com `adminClient` (`service_role`) em quatro tabelas usando ids que vieram direto do corpo: `documentVersionId`, `addendumId`, `contractId` e `dealId`. Nenhum é conferido contra a organização do chamador — o `organizationId` até é exigido na linha 60, mas nunca é usado para validar coisa alguma. Um usuário autenticado de qualquer tenant sobrescreve `signature_token`, `signature_status` e `signature_url` de contratos alheios. Na action 'status' o problema é de leitura: `signatureToken` vem do corpo e a função consulta a API do ZapSign com o token da conta corporativa, devolvendo `signers` e a URL do arquivo assinado de qualquer documento daquela conta — vazamento de contrato assinado de outro cliente. A action 'webhook' (linhas 166-225) fecha o conjunto: qualquer autenticado marca um contrato como SIGNED informando o token.

IMPACTO Adulteração do estado de assinatura de contratos de outros tenants e leitura de documentos assinados alheios via ZapSign.

CORREÇÃO SUGERIDA Validar associação do usuário a `organizationId`; carregar a linha alvo e conferir que seu `organization_id` bate com o do chamador antes de qualquer update; na action 'status', localizar primeiro a linha local que possui aquele `signature_token` dentro da organização do usuário e só então consultar o ZapSign. A action 'webhook' deve sair desta função e virar endpoint próprio, autenticado por segredo compartilhado com o ZapSign.

COMO FOI VERIFICADO Arquivo lido integralmente; `organizationId` exigido na linha 60 e não referenciado em nenhuma consulta.

ALTA

C3-04 asaas-webhook falha aberto: sem a variável de ambiente, aceita qualquer POST

`supabase/functions/asaas-webhook/index.ts` linha(s) 28-36

```
const webhookToken = Deno.env.get('ASAAS_WEBHOOK_TOKEN') ?? '';
const incoming = req.headers.get('asaas-access-token') ?? '';
console.log(`[asaas-webhook] incoming=${mask(incoming)} expected=${mask(webhookToken)} ...`);
if (webhookToken && incoming !== webhookToken) {
  return json({ error: 'Invalid webhook token' }, 401);
}
```

POR QUE É EXPLORÁVEL A condição começa por “webhookToken &&”. Se ASAAS_WEBHOOK_TOKEN não estiver definida no ambiente do projeto — ou for string vazia, ou for removida numa troca de ambiente — a expressão curto-circuita e a validação inteira é pulada: a função passa a aceitar qualquer POST anônimo. Daí em diante o corpo é processado com service_role: PAYMENT_RECEIVED marca client_charges como paga, faz a baixa do recebível em internal_transactions (business_status RECEBIDO, status CONCILIATED) e insere lançamento de taxa; BILL_PAID marca supplier_payments e boletos como pagos. O identificador precisa apenas casar com um asaas_payment_id/asaas_bill_id existente. É uma falha fail-open: o modo inseguro é o default silencioso. A linha 33 ainda registra em log um prefixo e um sufixo do token esperado, junto do comprimento — vazamento parcial de segredo no log.

IMPACTO Baixa fraudulenta de contas a receber e a pagar por requisição anônima, caso a variável não esteja configurada.

CONDIÇÃO DE EXPLORABILIDADE Explorabilidade condicionada a ASAAS_WEBHOOK_TOKEN ausente ou vazia no ambiente do projeto Supabase.

CORREÇÃO SUGERIDA Inverter para fail-closed: se webhookToken estiver vazio, responder 503 e não processar nada. Comparar em tempo constante. Remover o log das linhas 32-33 ou reduzi-lo a um booleano de match. Adicionar validação de inicialização que recuse subir sem a variável.

COMO FOI VERIFICADO Arquivo lido integralmente; condição de guarda na linha 34.

MÉDIA

C3-05 asaas-charge action 'resend' envia boleto de outro tenant para e-mail escolhido pelo atacante

supabase/functions/asaas-charge/index.ts linha(s) 84-115

```
const { data: ch } = await admin.from('client_charges')
  .select('asaas_payment_id,billing_type,party_email,status')
  .eq('id', chargeId).eq('organization_id', organization_id).maybeSingle();

const emailOverride = body.email ?? ch.party_email;
if (emailOverride) sendBody.emails = [emailOverride];
await fetch(`${asaasBase}/payments/${ch.asaas_payment_id}/sendByMail`, { ... });
```

POR QUE É EXPLORÁVEL Consequência direta de C2-03. Como organization_id não é validado contra o chamador, o par (charge_id, organization_id) do tenant alvo satisfaz os dois .eq(). O campo email do corpo então sobrepõe o destinatário legítimo e o Asaas envia a segunda via do boleto — com valor, vencimento, dados do sacado e linha digitável — para o endereço do atacante. É exfiltração de dado financeiro de outro tenant por um canal que não passa pela RLS.

IMPACTO Exfiltração de boletos e dados de cobrança de outro tenant para endereço arbitrário.

CORREÇÃO SUGERIDA Corrigir C2-03 (checagem de associação) e, adicionalmente, restringir emailOverride aos endereços já cadastrados daquele cliente, em vez de aceitar qualquer string.

COMO FOI VERIFICADO Arquivo lido integralmente; caminho de 'resend' nas linhas 84-115.

MÉDIA

C3-06 notify-broker-proposal e notify-opportunity-interest não verificam Authorization nenhuma

supabase/functions/notify-broker-proposal/index.ts linha(s) 22-40 (e
 notify-opportunity-interest/index.ts:28-53)

```
serve(async (req: Request) => {
  if (req.method === 'OPTIONS') return new Response('ok', { headers: corsHeaders });

  const supabaseUrl = Deno.env.get('SUPABASE_URL') ?? '';
  const serviceRoleKey = Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') ?? '';
  // nenhuma leitura de req.headers.get('Authorization') em todo o arquivo
  const { proposalId, organizationId } = await req.json();
```

POR QUE É EXPLORÁVEL Ao contrário das demais, estas duas funções não leem o header Authorization em momento algum. Elas filtram corretamente por organizationId nas consultas (o que impede montar notificação com dados de outro tenant — ponto positivo), mas qualquer um que conheça um par válido de ids dispara notificação in-app e e-mail para todos os owners/admins daquela organização, quantas vezes quiser. Serve como amplificador de spam e de phishing interno, já que o conteúdo do e-mail vem de campos gravados por formulário público (ver C5-03). Ainda que o verify_jwt da plataforma esteja ligado, ele é satisfeito pela chave anon, que é pública.

IMPACTO Spam ilimitado de notificação e e-mail para administradores, com conteúdo parcialmente controlado pelo atacante.

CORREÇÃO SUGERIDA Adotar o gate já usado nas funções de cron (comparar Authorization com Bearer <service_role>) e chamar estas funções a partir de trigger/serviço, ou exigir sessão válida com associação a organizationId. Somar rate limiting por (organizationId, interestId).

COMO FOI VERIFICADO Dois arquivos lidos integralmente; ausência de qualquer leitura de Authorization.

MÉDIA

C3-07 partner-portal-upload grava em bucket público com contractId arbitrário e content-type do usuário

supabase/functions/partner-portal-upload/index.ts linha(s) 88-102

```
if (target === 'invoice') {
  if (!contractId) return json({ error: 'contractId é obrigatório ...' }, 400);
  const path = `invoices/${contractId}/${Date.now()}_${safeName}`;
  const { error: uploadError } = await admin.storage.from('documents')
    .upload(path, bytes, { contentType: file.type || 'application/octet-stream', ... });
  const { data: pub } = admin.storage.from('documents').getPublicUrl(path);
  return json({ publicUrl: pub.publicUrl });
```

POR QUE É EXPLORÁVEL O ramo target='invoice' nunca confere que contractId pertence ao workspace do token validado — diferente do ramo padrão, que usa o workspaceId derivado do token no caminho. Um parceiro qualquer grava arquivo na pasta de qualquer contrato. O bucket documents é público (confirmado em storage.buckets: public=true, allowed_mime_types NULL, file_size_limit NULL) e a função aceita o content-type informado pelo cliente, então dá para publicar text/html arbitrário numa URL do domínio de storage da organização — útil para phishing e para hospedar carga maliciosa com ar de legitimidade. Não há limite de tipo nem de tamanho no bucket.

IMPACTO Contaminação de anexos de contratos alheios e hospedagem de conteúdo arbitrário (inclusive HTML) em URL pública da organização.

CORREÇÃO SUGERIDA Validar que contractId pertence ao workspace do token antes de montar o caminho; forçar allowlist de content-type (application/pdf e imagens) em vez de confiar em file.type; definir allowed_mime_types e file_size_limit no bucket documents; avaliar torná-lo privado com URL assinada, no padrão já adotado para opura-docs.

COMO FOI VERIFICADO Arquivo lido integralmente; flag public=true confirmada em storage.buckets no banco remoto.

C4 Chaves e segredos expostos

BAIXA

C4-01 Chave publishable e project ref do Supabase embutidos em script versionado

check_suppliers.js linha(s) 3-4

```
const supabaseUrl = 'https://oxedkknreghxrgenyjiu.supabase.co';  
const supabaseKey = 'sb_publishable_IgIC72BIXcLNix4ARLo0QA_0UGDrnzW';  
const supabase = createClient(supabaseUrl, supabaseKey);
```

POR QUE É EXPLORÁVEL A chave publishable/anon é desenhada para ser pública — ela já vai no bundle do frontend — então o vazamento em si não é o problema. O que importa é o efeito combinado: o arquivo está versionado (git ls-files confirma) e dá a qualquer leitor do repositório um cliente pronto, apontado para o projeto de produção, sem passo nenhum de configuração. Foi exatamente com essa chave que o achado C1-02 foi confirmado. Os demais scripts da raiz (test_db.js, test_insert.js, unlink_db.js, query_doc.js) fazem o certo: leem de process.env ou do .env. Este é o único com valor cravado. Registra-se que nenhum segredo real — service_role, JWT, chave de API — foi encontrado em conteúdo versionado.

IMPACTO Reduz a zero o atrito para explorar qualquer falha de RLS no projeto de produção.

CORREÇÃO SUGERIDA Trocar os literais por process.env, como nos demais scripts; ou mover check_suppliers.js para _dev_scripts/ (já no .gitignore) e remover do índice do git.

COMO FOI VERIFICADO git ls-files confirma o arquivo versionado; git grep por padrões de segredo em todo o conteúdo versionado não encontrou segredo real.

BAIXA

C4-02 Placeholders de segredo como fallback de COALESCE nos agendamentos pg_cron

supabase/migrations/20260224000002_setup_billing_cron.sql linha(s) 37 (e 20261118000011_task_alert_sent_at.sql:31-35)

```
'Authorization', 'Bearer ' || (SELECT COALESCE(current_setting('vault.service_role_key', true),  
  'INTERNAL_SECRET_HERE'))  
  
-- 20261118000011_task_alert_sent_at.sql:  
'Authorization', 'Bearer ' || coalesce(  
  current_setting('vault.service_role_key', true),  
  current_setting('app.service_role_key', true),  
  'CONFIGURE_SERVICE_ROLE_KEY')
```

POR QUE É EXPLORÁVEL O padrão COALESCE(variável, 'PLACEHOLDER') faz o literal virar o segredo efetivo quando a variável não está configurada. Aqui o efeito imediato é de disponibilidade, não de exposição: as funções alvo comparam o header com Bearer <service_role> e recusam o placeholder, então o cron de faturamento e o de alerta de tarefa falham 401 em silêncio — ninguém é avisado de que pararam. O risco de segurança é o inverso do costume: como o valor certo precisa ser colado à mão (o comentário da linha 47 manda literalmente substituir pela service_role key), o próximo passo natural de quem for consertar é cravar a chave real numa migration versionada. Não há validação de inicialização que recuse o placeholder.

IMPACTO Jobs de cron falhando silenciosamente e convite estrutural a commitar a service_role key numa migration.

CORREÇÃO SUGERIDA	Remover os fallbacks literais e deixar a chamada falhar de forma visível quando a chave não estiver no Vault; ler o segredo apenas de vault.decrypted_secrets, como a migration 20260514000002 (quality-sla-enforcement) já faz corretamente; e acrescentar teste que recuse migration contendo os literais INTERNAL_SECRET_HERE / CONFIGURE_SERVICE_ROLE_KEY / SUA_SERVICE_ROLE_KEY.
COMO FOI VERIFICADO	Migrations lidas na íntegra; padrão correto identificado em 20260514000002_quality_sla_cron.sql:21.

INFORMATIVA**C4-03 Project ref do Supabase cravado como fallback de Access-Control-Allow-Origin**

supabase/functions/process-billing-ruler/index.ts linha(s) 8-11

```
const corsHeaders = {
  'Access-Control-Allow-Origin': Deno.env.get('FRONTEND_URL') ??
  'https://oxedkknreghxrgenyjiu.supabase.co',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
}
```

POR QUE É EXPLORÁVEL	Não é credencial e não concede acesso: o project ref é visível em qualquer requisição do app. Fica o registro como higiene — identificador de ambiente de produção cravado no código, que vira armadilha em fork, em homologação ou numa eventual migração de projeto. Como toda produção real define FRONTEND_URL, o fallback nunca deveria ser exercido.
IMPACTO	Nenhum impacto direto de segurança; risco de configuração errada silenciosa entre ambientes.
CORREÇÃO SUGERIDA	Falhar explicitamente quando FRONTEND_URL não estiver definida, em vez de cair num literal de produção.
COMO FOI VERIFICADO	Arquivo lido integralmente.

C5 Inputs sem tratamento (XSS)**ALTA****C5-01 XSS armazenado na Academia: qualquer membro injeta HTML servido a todos os matriculados**

components/academy/AcademyLessonPlayer.tsx linha(s) 180-183

```
case 'TEXT0':
  return (
    <div
      className="prose prose-sm max-w-none ..."
      dangerouslySetInnerHTML={{ __html: lesson.conteudo_html || '' }}
    />
  );
```

POR QUE É EXPLORÁVEL	conteudo_html vem da tabela academy_lessons e é injetado sem qualquer sanitização. As policies de escrita dessa tabela são academy_lessons_insert e academy_lessons_update, ambas com expressão is_org_member(org_id) — ou seja, QUALQUER membro da organização, no papel mais baixo, escreve o HTML. O conteúdo é depois renderizado para todo colaborador matriculado que abrir a aula, incluindo owners e admins. Não existe biblioteca de sanitização no projeto: package.json não declara DOMPurify nem equivalente, e não há utilitário próprio de escape. Um membro comum publica aula com e executa script na sessão de um administrador — o token do Supabase é acessível ao JavaScript da página, então o resultado é sequestro de sessão privilegiada. É escalada de privilégio dentro do tenant.
-----------------------------	--

IMPACTO	Execução de script na sessão de administradores; roubo de sessão e escalada de privilégio dentro da organização.
CORREÇÃO SUGERIDA	Adicionar dependência de sanitização (DOMPurify) e criar um helper único — por exemplo <code>utils/sanitizeHtml.ts</code> — com <code>allowlist</code> de tags e atributos, proibindo event handlers e javascript: em <code>href/src</code> . Aplicar em todo <code>dangerouslySetInnerHTML</code> que renderize dado de banco. Sanitizar também na escrita, para não depender só da leitura.
COMO FOI VERIFICADO	Sink lido no arquivo; policies de <code>academy_lessons</code> confirmadas em <code>pg_policies (is_org_member)</code> ; ausência de lib de sanitização confirmada em <code>package.json</code> .

ALTA**C5-02 Template de contrato injetado via innerHTML na geração de PDF****components/ContractDetailView.tsx** linha(s) 885-893

```
const rendered = renderTemplate(template.body_html, varMap);

const container = window.document.createElement('div');
container.style.cssText = 'position:fixed;left:-9999px;top:0;width:794px;...';
container.innerHTML = rendered;
window.document.body.appendChild(container);
```

POR QUE É EXPLORÁVEL `body_html` vem de `contract_templates`, cuja policy de escrita é `contract_templates_org` com `is_org_member(organization_id)` — de novo, qualquer membro da organização. O HTML é atribuído via `innerHTML` a um elemento que é efetivamente anexado ao documento (linha 893), no contexto de quem gera o PDF, tipicamente um usuário do financeiro ou administrador. `innerHTML` não executa `<script>`, mas executa handlers de erro e de carga: `<img src=x onerror=...>` e `<svg onload=...>` disparam normalmente. O container fica fora da tela, então a vítima não vê nada acontecer. Não há sanitização em `renderTemplate` nem no ponto de uso.

IMPACTO	Execução de script na sessão de quem gera o PDF do contrato, disparada por qualquer membro da organização.
CORREÇÃO SUGERIDA	Sanitizar <code>rendered</code> com o mesmo helper de C5-01 antes da atribuição; aplicar também nos dois pontos de preview de template (C5-04).
COMO FOI VERIFICADO	Sink lido no arquivo; policy <code>contract_templates_org</code> confirmada em <code>pg_policies</code> .

MÉDIA**C5-03 E-mails HTML montados por interpolação, com campo vindo de formulário público anônimo****supabase/functions/notify-opportunity-interest/index.ts** linha(s) 134-174 (e `notify-broker-proposal/index.ts:128-149`)

```
<p style="...">${opp.title}</p>
${opp.subtitle ? `<p style="...">${opp.subtitle}</p>` : ''}
<span style="...">${interest.contact_name}</span>
<a href="mailto:${interest.contact_email}" style="...">${interest.contact_email}</a>
<p style="...">${interest.message}</p>
```

POR QUE É EXPLORÁVEL O HTML do e-mail é montado por template string, sem nenhum escape. Os campos `contact_name`, `contact_email`, `contact_phone` e `message` vêm de `opportunity_interests`, alimentada pelo formulário público de manifestação de interesse (RPC `fn_investor_portal_submit_interest`, confirmada como executável por `anon`) — ou seja, são texto de atacante anônimo. Em `notify-broker-proposal` o mesmo vale para `notes`, `buyer_name` e `broker_email`. O destinatário é a caixa de todos os `owners/admins` da organização. Clientes de e-mail modernos bloqueiam script, então não é XSS clássico; o que se consegue é quebrar a estrutura do HTML e injetar conteúdo e âncoras arbitrárias — um `<a href>` convincente dentro de uma mensagem que chega do domínio corporativo da própria empresa. Combinado com C3-06 (as duas funções não exigem credencial), o atacante escolhe a hora e a frequência do disparo. `contact_email` ainda vai para dentro de um atributo `href`, o contexto mais frouxo dos presentes.

IMPACTO Phishing dirigido a administradores, com conteúdo controlado pelo atacante, entregue pelo domínio corporativo.

CORREÇÃO SUGERIDA Criar helper de escape de HTML (e um específico para contexto de atributo) em `supabase/functions/_shared/` e aplicar a toda interpolação de dado do banco nas duas funções. Validar `contact_email` como e-mail antes de usá-lo em `href`, e truncar `message` a um tamanho máximo.

COMO FOI VERIFICADO Duas funções lidas integralmente; `fn_investor_portal_submit_interest` confirmada como executável por `anon` em `pg_proc`.

BAIXA

C5-04 Preview de template renderiza HTML do próprio autor sem sanitização (self-XSS)

`components/ContractTemplateManager.tsx` linha(s) 248-252 (e `components/DunningModule.tsx:243-246`)

```
{previewMode ? (
  <div
    className="min-h-[480px] ... prose prose-sm max-w-none"
    dangerouslySetInnerHTML={{ __html: previewContent }}
  />
) : ( ... )}
```

POR QUE É EXPLORÁVEL Nos dois casos o HTML renderizado é o que o próprio usuário acabou de digitar no `textarea` ao lado (`previewContent` vem de `renderTemplate(bodyHtml, ...)` na linha 191; `previewBody` vem de `form.body_template` na linha 101 do `DunningModule`). Sozinho isso é self-XSS: a vítima teria de colar o payload em si mesma, o que não configura vulnerabilidade explorada por terceiro. Fica registrado como baixa por dois motivos: é o mesmo dado que, depois de salvo, alimenta o sink de C5-02 e o corpo dos e-mails de cobrança; e a correção é a mesma linha de código, então separar as duas passadas só cria retrabalho.

IMPACTO Sem impacto direto isolado; relevante como parte da mesma superfície de C5-02.

CORREÇÃO SUGERIDA Aplicar o helper de sanitização de C5-01 nos dois pontos de `preview`.

COMO FOI VERIFICADO Origem de `previewContent` e `previewBody` rastreada até o estado local do formulário.

6. Recomendações priorizadas

A ordem não é por severidade isolada, é por quanto risco cada correção remove por unidade de esforço. P1 são as quatro mudanças que, sozinhas, fecham a quebra de multi-tenant; nenhuma delas exige refatoração.

P1 Imediato (hoje)		
1.	Substituir a policy de INSERT de organization_members por is_org_manager(organization_id) OR is_superuser(), movendo o auto-vínculo do primeiro dono para uma RPC dedicada	C1-01
2.	REVOKE EXECUTE FROM PUBLIC, anon em client_portal_generate_token e broker_portal_generate_token, e exigir is_org_manager dentro das duas funções	C3-01
3.	Reescrever as policies de invoices com recorte por organização e remover as duas policies anon	C1-02
4.	Completar a regra de is_shared nas policies de clients, suppliers e partner_workspaces — dizer com QUEM o registro é compartilhado, ou remover a perna até haver regra correta	C1-05
5.	Trocar employeeid por token em labor-portal-ged-download, no desenho de academy-portal-media	C3-02
P2 Esta semana		
1.	Criar supabase/functions/_shared/auth.ts com checagem de associação e papel, e aplicar em asaas-charge, asaas-payment, sign-contract, send-bi-report e sinapi-import	C2-01, C2-02, C2-03, C3-05
2.	Adicionar checagem de posse por objeto em sign-contract (contrato, aditivo, versão, deal) e escopar a action 'status'	C3-03
3.	Tornar o asaas-webhook fail-closed e tirar o token do log	C3-04
4.	Adicionar DOMPurify, criar utils/sanitizeHtml.ts e aplicar nos quatro sinks do frontend	C5-01, C5-02, C5-04
5.	Autenticar notify-broker-proposal e notify-opportunity-interest e escapar as interpolações de HTML dos e-mails	C3-06, C5-03
P3 Este mês		
1.	Varrer pg_proc atrás de toda SECURITY DEFINER com ACL herdando PUBLIC e revogar; adicionar o REVOKE ao template de migration	C3-01
2.	Remover as policies anon remanescentes de opura_cno_areas e opura_cno_reductions e escopar payment_types	C1-03, C1-04
3.	Validar contractId e restringir content-type em partner-portal-upload; definir allowed_mime_types e file_size_limit no bucket documents	C3-07
4.	Eliminar os placeholders de segredo das migrations de cron e ler do Vault	C4-02
5.	Tirar do git a credencial cravada em check_suppliers.js e o project ref de process-billing-ruler	C4-01, C4-03

P4 Contínuo		
1.	Transformar cada achado em verificação mecânica no CI, ao lado de __tests__/orgContextGuard.test.ts: teste que recusa policy com qual=true para anon, teste que recusa SECURITY DEFINER com ACL PUBLIC, teste que recusa dangerouslySetInnerHTML sem passar pelo helper de sanitização	todos
2.	Adotar revisão obrigatória para toda migration que crie policy ou função SECURITY DEFINER	C1, C3

7. Issues para o GitHub

Cada bloco abaixo é o texto completo de uma issue em Markdown, pronto para copiar e colar. Achados triviais do mesmo tema foram agrupados numa issue única para não gerar spam — por exemplo, os quatro sinks de HTML sem sanitização viram uma issue só, porque a correção é a mesma peça de código. São **15 issues** para **22 achados**.

--- ISSUE 1 ---

```
## [Segurança] Policy de INSERT em organization_members permite auto-promoção a owner de qualquer organização

**Labels:** `security`, `severidade:critica`, `multi-tenant`

### Problema

organization_members é a tabela que define quem pertence a que organização e com que papel. Toda a RLS do sistema depende dela: is_org_member(org) e is_org_manager(org) são SECURITY DEFINER e resolvem a permissão consultando exatamente essa tabela, casando por auth.uid() ou pelo e-mail do JWT. A policy de INSERT tem WITH CHECK (true) — não restringe organization_id, não restringe email e não restringe role. Qualquer conta autenticada (inclusive uma recém-criada por self-signup, que está habilitado em supabase/config.toml) executa um único INSERT informando o próprio e-mail, o organization_id alvo e role='owner'. A partir daí is_org_member e is_org_manager retornam TRUE para aquela organização, e todas as policies org-scoped do sistema — financeiro, folha, contratos, documentos — passam a liberar leitura e escrita. Quebra total do multi-tenant e escalada de privilégio numa única requisição.

COMPROVADO EM PRODUÇÃO, em transação abortada. Assumindo o papel authenticated com um e-mail sem nenhum vínculo, o INSERT com role='owner' foi ACEITO pela RLS, e as medições antes/depois na mesma transação foram: vínculos do ator 0 → 1; is_org_member FALSE → TRUE; is_org_manager FALSE → TRUE; linhas visíveis em organizations 0 → 1; em internal_transactions 0 → 2214. Ou seja: de nenhum acesso a 2.214 lançamentos financeiros e direitos de proprietário, com uma instrução. Nada foi persistido — o bloco termina em RAISE EXCEPTION, que aborta a transação por construção.

### Evidência

`supabase/migrations/20260215000009_fix_org_rls_recursive.sql` (linhas 65-67):

...
create policy "Authenticated users can create memberships"
on organization_members for insert to authenticated
with check (true);
...

_Verificado assim:_ Definição lida em pg_policies no banco remoto; cadeia confirmada em pg_get_functiondef de is_org_member e is_org_manager; e exploração executada de ponta a ponta contra o banco de produção dentro de BEGIN ... RAISE EXCEPTION (rollback garantido, zero resíduo).

### Impacto

- Tomada de controle completa de qualquer tenant do SaaS por qualquer usuário cadastrado que conheça o UUID da organização alvo.

> **Condição de explorabilidade (C1-01):** O atacante precisa conhecer o organization_id (UUID) do alvo: a RLS de organizations impede listar as organizações, e a primeira execução da prova, sem o UUID, inseriu organization_id NULL e não escalou nada. Isso reduz a superfície, mas não é um controle de segurança — o UUID não é segredo: ele viaja no link de convite (invite-member monta redirectTo como /?org=<uuid>), aparece na URL da aplicação e é conhecido por qualquer pessoa que já tenha sido membro. O cenário realista é o ex-funcionário removido que anotou o UUID e se readiciona como owner.

### Correção sugerida

Trocar o WITH CHECK (true) por uma condição que só permita a linha que o fluxo legítimo precisa, e mover a criação de membro para o servidor. Concretamente: (a) restringir a policy a is_org_manager(organization_id) OR is_superuser(); (b) para o auto-vínculo do primeiro dono ao criar a organização, usar uma RPC SECURITY DEFINER dedicada que só aceite organização sem nenhum membro; (c) o convite de terceiros já tem caminho correto e checado — a Edge Function invite-member.

### Critérios de aceite
```

- [] A policy "Authenticated users can create memberships" não existe mais em pg_policies
- [] Usuário não-membro recebe erro de RLS ao tentar INSERT em organization_members (teste automatizado)
- [] Criar organização nova continua funcionando e o criador vira owner
- [] Convite via invite-member continua criando a linha de membership
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] RPCs de credencial de portal executáveis por anon – três sem guarda nenhuma

Labels: `security`, `severidade:critica`, `multi-tenant`

Problema

São OITO as RPCs de credencial de portal com a ACL aberta ao papel anon – os seis emissores (client, broker, investor, partner, supplier e o do colaborador) mais dois revogadores (partner e supplier), que permitiriam a um anônimo REVOGAR o acesso de um parceiro legítimo.

■ **CORREÇÃO DE ESCOPO (2026-09-02):** a primeira versão deste achado afirmava que as oito emitiam token 'sem verificar quem chama'. A leitura de pg_get_functiondef das oito, feita ao escrever a correção, mostrou que isso vale para apenas TRÊS: client_portal_generate_token, broker_portal_generate_token e portal_generate_token (colaborador). As outras cinco já chamavam <portal>_portal_can_manage_tokens(p_org_id), que exige organization_members.role IN ('owner', 'admin') e confere que o titular pertence à organização. Como anon não tem auth.uid() nem auth.jwt(), essa guarda já as tornava inalcançáveis anonimamente, apesar da ACL aberta. O vetor anônimo real existia em três, não em oito – e a prova de conceito abaixo explorou justamente uma delas. O REVOKE segue necessário nas oito: a ACL aberta é defeito por si só, e é o que separava as cinco corretas de uma linha de código.

As três desprotegidas eram SECURITY DEFINER (rodam como owner, ignorando RLS), recebiam o id do titular como parâmetro e emitiam um token de portal válido por 90 dias – sem verificar quem chama, sem conferir que o titular pertence a p_org_id e sem exigir sequer sessão. As migrations fazem GRANT EXECUTE TO authenticated, mas nunca fazem REVOKE EXECUTE FROM PUBLIC; como o PostgreSQL concede EXECUTE a PUBLIC por padrão, a ACL efetiva no banco remoto é {X/postgres, postgres=X, anon=X, authenticated=X, service_role=X} – o “=X” inicial é o PUBLIC, e anon aparece explicitamente. O papel anon executa. A cadeia completa é: chamar client_portal_generate_token, receber um token válido, chamar client_portal_get_data(token) (também SECURITY DEFINER e anon) para ler contratos, parcelas e avisos daquele cliente, e chamar a Edge Function portal-ged-download com o mesmo token para baixar os documentos do GED. O ON CONFLICT DO UPDATE ainda substitui o token legítimo, derrubando o acesso do cliente real – negação de serviço como efeito colateral.

COMPROVADO EM PRODUÇÃO, em transação abortada. Assumindo o papel anon (exatamente o que a chave publicada no bundle concede) e sem login nenhum: a leitura direta da tabela clients retornou 0 linhas – a RLS funciona –, mas client_portal_generate_token(<client_id>, <org_id>) executou e devolveu um token UUID válido; em seguida client_portal_get_data(<token>) devolveu o payload do portal com "valid": true e os dados cadastrais do cliente, incluindo nome completo e CPF (redigido neste relatório). Ou seja, a RPC de emissão de credencial contorna por completo a RLS que protege a tabela. Nada foi persistido: o bloco termina em RAISE EXCEPTION, então o token gerado foi descartado e o token legítimo do cliente permanece intacto.

Evidência

`supabase/migrations/20261128000001\_client\_portal\_tokens.sql` (linhas 69-89 (e 20261224000001_broker_portal_tokens.sql:42-66)):

...

```
CREATE OR REPLACE FUNCTION public.client_portal_generate_token(p_client_id uuid, p_org_id uuid)
  RETURNS text LANGUAGE plpgsql SECURITY DEFINER AS $function$
```

```
DECLARE v_token TEXT;
```

```
BEGIN
```

```
  v_token := gen_random_uuid()::text;
```

```
  INSERT INTO public.client_portal_tokens (org_id, client_id, token)
```

```
  VALUES (p_org_id, p_client_id, v_token)
```

```
  ON CONFLICT (client_id) DO UPDATE SET token = v_token, ...;
```

```
  RETURN v_token;
```

```
END; $function$
```

```
GRANT EXECUTE ON FUNCTION public.client_portal_generate_token(UUID, UUID) TO authenticated;
```

```
-- nunca há REVOKE EXECUTE ... FROM PUBLIC
```

```

...

_Verificado assim:_ pg_get_functiondef e pg_proc.proacl lidos no banco remoto (ACL efetiva confirma
anon=X em ambas); e cadeia emissão → leitura executada como papel anon contra o banco de produção,
dentro de BEGIN ... RAISE EXCEPTION (rollback garantido, token descartado).

### Impacto

- Sequestro anônimo dos Portais do Cliente, do Corretor e do Colaborador (as três sem guarda). Nas
  outras cinco, a ACL aberta era defeito latente – a guarda interna as segurava.

### Correção sugerida

Duas correções, ambas necessárias. (1) REVOKE EXECUTE ON FUNCTION ... FROM PUBLIC, anon nas oito
funções, e varrer as demais SECURITY DEFINER pelo mesmo defeito de ACL. (2) Adicionar autorização
dentro da função: exigir is_org_manager(p_org_id) e validar que o id do titular pertence a p_org_id –
emitir credencial de acesso é operação administrativa e não deve depender só do GRANT.

### Critérios de aceite

- [ ] has_function_privilege('anon', ...) retorna false para as oito funções
- [ ] Chamada da RPC com a chave anon retorna erro de permissão
- [ ] Chamada por autenticado que não é gestor da organização retorna erro
- [ ] Gerar link do Portal do Cliente/Corretor pela tela de admin continua funcionando
- [ ] Varredura de pg_proc confirma que nenhuma SECURITY DEFINER sensível tem ACL com PUBLIC
- [ ] Existe teste automatizado que falha se a condição voltar

```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

```

## [Segurança] Tabela invoices legível e gravável por anônimo e sem isolamento entre tenants

**Labels:** `security`, `severidade:critica`, `multi-tenant`

### Problema

Estado atual no banco remoto: três policies em invoices – “Suppliers can view their own invoices”
(SELECT, papel anon, USING true), “Suppliers can insert their own invoices” (INSERT, papel anon, WITH
CHECK true) e “invoices_authenticated_all” (ALL, authenticated, USING/CHECK true). Nenhuma tem
recorte: o nome fala em “their own”, mas a expressão é literalmente true. A tabela sequer possui
coluna organization_id – o vínculo com o tenant é indireto, por supplier_id. A chave anon fica
publicada no bundle JavaScript por construção, então a policy anon equivale a acesso público.
Verificado ao vivo: GET /rest/v1/invoices com a chave anon do .env, sem qualquer login, retornou HTTP
206 e Content-Range 0-828/829 – 829 notas fiscais de fornecedor de todos os tenants, com nome de
arquivo, fornecedor e caminho no storage. O INSERT anon com CHECK true também permite injetar notas
forjadas.

### Evidência

`supabase/migrations/20260516100000_fix_invoices_qls_boletos.sql` (linhas 14-24):
...
-- Política única: membros autenticados de qualquer organização têm acesso total
CREATE POLICY "invoices_authenticated_all" ON public.invoices
  FOR ALL TO authenticated
  USING (true)
  WITH CHECK (true);
...

_Verificado assim:_ Confirmado por requisição HTTP real (HTTP 206, Content-Range 0-828/829) com a chave
anon pública.

### Impacto

- Vazamento público de 829 notas fiscais (fornecedor, valor, vencimento, centro de custo, caminho do
  arquivo) e inserção de notas falsas por anônimo.

### Correção sugerida

Remover as duas policies anon e substituir invoices_authenticated_all por policies com recorte real.
Como a tabela não tem organization_id, o caminho mais seguro é (a) acrescentar a coluna com backfill

```

a partir de `suppliers/purchase_orders`, ou (b) enquanto isso, escrever a policy via EXISTS sobre `suppliers` com `is_org_member(suppliers.organization_id)`. O Portal do Fornecedor não precisa da policy anon: ele já passa pelas Edge Functions `supplier-portal-download` e `supplier-portal-upload`, que usam `service_role` após validar o token.

Critérios de aceite

- [] GET `/rest/v1/invoices` com a chave anon retorna lista vazia
- [] Usuário autenticado do tenant A não vê nenhuma nota do tenant B
- [] POST anon em `/rest/v1/invoices` é recusado por RLS
- [] Portal do Fornecedor continua listando e enviando notas normalmente
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] A perna “OR is_shared” das policies de leitura vaza 127 cadastros entre tenants

****Labels:**** `security`, `severidade:alta`, `multi-tenant`

Problema

A intenção do sinalizador `is_shared` é permitir que um cadastro seja reaproveitado entre as organizações de um mesmo grupo. Mas a expressão escrita não tem a segunda metade da regra: “OR is_shared” é verdadeiro sozinho, sem nenhuma condição sobre quem está lendo. Não é “compartilhado com as organizações do grupo” – é compartilhado com TODOS os usuários autenticados do SaaS inteiro, inclusive clientes concorrentes que nunca tiveram relação com aquele tenant. O mesmo padrão está em três policies: `clients`, `suppliers` (“Users can view suppliers of their organization” – o nome contradiz a expressão) e `partner_workspaces`.

COMPROVADO EM PRODUÇÃO (somente leitura). Assumindo o papel `authenticated` com um e-mail sem nenhuma linha em `organization_members`, a contagem de linhas visíveis foi: `clients` 7, `suppliers` 119, `partner_workspaces` 1 – enquanto `organizations` retornou 0, o que prova que a identidade usada realmente não tinha vínculo nenhum. São 119 de 244 fornecedores (49% da base) e 7 de 46 clientes expostos a qualquer conta do SaaS. Os registros de `clients` carregam PII: nome, CPF/CNPJ, endereço e telefone.

Evidência

``supabase/migrations/aplicar_20270914000021_org_not_null_e_policies_is_shared.sql`` (linhas 67-69):

```
...
CREATE POLICY "Allow authenticated users to read clients"
ON public.clients FOR SELECT TO authenticated
USING (public.is_org_member(organization_id) OR is_shared);
...
```

Verificado assim: `pg_policies` (busca por qual LIKE `'%is_shared%'`) no banco remoto, contagem de linhas `is_shared` por tabela, e leitura executada como papel `authenticated` sem vínculo, em transação abortada.

Impacto

- Vazamento cross-tenant de 127 cadastros – incluindo CPF/CNPJ, endereço e telefone de pessoas físicas – para qualquer usuário autenticado de qualquer organização do SaaS.

> ****Condição de explorabilidade (C1-05):**** O impacto REAL hoje é baixo e o defeito é uma bomba-relógio, não um vazamento em curso: as quatro organizações existentes no banco pertencem todas ao mesmo cliente (Alpa Construtora, ALPA Empreendimentos, a SPE Garden Cambuhy e uma organização pessoal), então os 127 registros circulam hoje apenas dentro do grupo que os possui. O defeito vira vazamento real no instante em que o segundo cliente entrar no SaaS – e é por isso que a correção é pré-requisito de onboarding, não item de manutenção. Registrado assim para não superdimensionar o presente nem subdimensionar o risco.

Correção sugerida

Completar a regra: o compartilhamento precisa dizer COM QUEM. Substituir “OR is_shared” por uma condição que exija que o leitor pertença ao mesmo grupo/holding do dono do registro – por exemplo “OR (`is_shared AND is_org_member(<org do grupo>)`)”, ou uma tabela explícita de compartilhamento (`organization_id` dono, `organization_id` destino). Aplicar às três policies. Enquanto a regra correta não existir, o caminho seguro é remover a perna `is_shared`.

Critérios de aceite

- [] Usuário sem vínculo em organization_members lê 0 linhas de clients, suppliers e partner_workspaces
- [] Usuário de uma organização do grupo continua vendo os cadastros compartilhados do grupo
- [] Usuário de organização fora do grupo não vê nenhum registro is_shared
- [] As três políticas (clients, suppliers, partner_workspaces) usam a mesma regra revisada
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] Portal do Colaborador exposto: anon lê folha de pagamento só com o UUID do colaborador

Labels: `security`, `severidade:critica`, `multi-tenant`

Problema

A Edge Function não valida token nem sessão: o único controle é que o employeeId informado exista na tabela employees. Mas ela é apenas a ponta visível. A varredura de pg_proc mostrou que o Portal do Colaborador INTEIRO funciona assim: as sete funções de leitura (portal_employee_summary, portal_get_payroll_runs, portal_get_documents, portal_get_ged_documents, portal_get_absences, portal_get_time_entries, portal_get_trainings) são SECURITY DEFINER, recebem p_employee_id cru e são executáveis pelo papel anon. components/LaborPortal.tsx:78-124 as chama exatamente assim, com o employeeId cru.

O próprio comentário da Edge Function admite o desenho ("a sessão do portal é anon/employeeId, sem token assinado nem login Supabase"), e a função irmã academy-portal-media documenta explicitamente, nas linhas 32-33, por que está errado: "O recorte vem do TOKEN (portal_tokens), não de um employeeId cru passado pelo cliente. Passar employeeId seria enumerável." Existe portal_tokens com employee_id, e existe até o padrão pronto do outro lado – fn_portal_get_ged_documents(p_token), usada pelo Portal do Cliente. A primitiva certa está escrita e não foi usada aqui.

COMPROVADO EM PRODUÇÃO (somente leitura, papel anon, sem login). O contraste é o que torna o achado inequívoco: anon NÃO tem sequer GRANT SELECT na tabela employees – a consulta direta falha com 42501 e a mensagem do Postgres chega a sugerir o GRANT que falta. Ainda assim, portal_employee_summary(<uuid>) devolveu o cadastro do colaborador (nome, cargo, status) e portal_get_payroll_runs(<uuid>) devolveu as FOLHAS DE PAGAMENTO, com competência e valores. As RPCs SECURITY DEFINER contornam por completo a proteção da tabela.

Evidência

`supabase/functions/labor-portal-ged-download/index.ts + 8 RPCs SECURITY DEFINER` (linhas 32-65 (e components/LaborPortal.tsx:78-124)):

...

// A ponta visível – a Edge Function:

```
const { employeeId, storagePath } = await req.json();
const { data: emp } = await admin.from('employees').select('id').eq('id', employeeId).maybeSingle();
if (empError || !emp) return json({ error: 'Colaborador não encontrado' }, 403);
// única checagem: o colaborador EXISTE. Sem token, sessão ou prova de identidade.
```

// A causa real – a camada de RPC, toda executável por anon e por employee_id cru:

```
// portal_employee_summary(p_employee_id)    portal_get_payroll_runs(p_employee_id)
// portal_get_documents(p_employee_id)       portal_get_ged_documents(p_employee_id)
// portal_get_absences(p_employee_id)        portal_get_time_entries(p_employee_id)
// portal_get_trainings(p_employee_id)       is_employee_shared_with_user(p_employee_id)
```

Verificado assim: pg_proc (prosecdef + has_function_privilege para anon) no banco remoto; chamadas executadas como papel anon contra produção, em transação abortada, devolvendo cadastro e folha de pagamento; e components/LaborPortal.tsx:78-124 confirmado passando employeeId cru.

Impacto

- Leitura anônima de folha de pagamento, ponto, ausências, treinamentos e documentos de RH de qualquer colaborador de qualquer tenant, bastando o UUID – que também pode ser mintado por anon via portal_generate_token (C3-01).

Correção sugerida

Criar as variantes fn_portal_*(p_token text) das sete funções, derivando o employee_id de portal_tokens

(validando `is_active` e `expires_at`) – mesmo desenho de `fn_portal_get_ged_documents(p_token)`, que já existe. Depois `REVOKE EXECUTE ... FROM PUBLIC`, anon nas sete antigas. Migrar `components/LaborPortal.tsx` para o token e aplicar o mesmo na Edge Function `labor-portal-ged-download`. Nenhum identificador de titular deve chegar cru pelo corpo da requisição.

Critérios de aceite

- [] `has_function_privilege('anon', ...)` é false para as sete RPCs por `p_employee_id`
- [] As novas `fn_portal_*(p_token)` devolvem os mesmos dados para token válido e erro para token expirado/inativo
- [] A Edge Function rejeita requisição sem token com 401
- [] Token de um colaborador não lê dado de outro
- [] O Portal do Colaborador abre por link com token e mostra folha, ponto, ausências e documentos
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Edge Functions confiam no `organization_id` do corpo da requisição sem checar associação

Labels: `security`, `severidade:alta`, `multi-tenant`

Problema

C2-03 - `organization_id` vem do corpo da requisição e nunca é conferido contra a associação do chamador

As três funções seguem o mesmo padrão: validam que existe um usuário, criam um cliente com `service_role` (que ignora a RLS) e passam a usar o `organization_id` vindo do corpo como se fosse confiável. Ele aparece em `.eq()`, o que dá a impressão de escopo – mas o escopo é o que o atacante escolheu, não o que ele tem direito. Um usuário do tenant A que conheça (ou enumere) um `transaction_id` do tenant B emite cobrança real no Asaas contra o recebível alheio, cancela cobranças do tenant B (action 'cancel', linhas 125-161) e reverte o status do recebível. Em `asaas-payment` o mesmo vale para `boleto_id` e `supplier_payment_id`, com efeito de dinheiro saindo. Como a RLS está fora do caminho, ela não serve de rede de segurança.

C3-05 - `asaas-charge` action 'resend' envia boleto de outro tenant para e-mail escolhido pelo atacante

Consequência direta de C2-03. Como `organization_id` não é validado contra o chamador, o par (`charge_id`, `organization_id`) do tenant alvo satisfaz os dois `.eq()`. O campo email do corpo então sobrepõe o destinatário legítimo e o Asaas envia a segunda via do boleto – com valor, vencimento, dados do sacado e linha digitável – para o endereço do atacante. É exfiltração de dado financeiro de outro tenant por um canal que não passa pela RLS.

Evidência

`supabase/functions/asaas-charge/index.ts` (linhas 47-73 (e `asaas-payment/index.ts`:70-89; `sign-contract/index.ts`:36-62)):

...

```
const { data: { user }, error: authError } = await userClient.auth.getUser();
if (authError || !user) return json({ error: 'Token inválido' }, 401);
```

```
const admin = createClient(supabaseUrl, serviceRoleKey, { ... }); // ignora RLS
const { organization_id, transaction_id } = body;
if (!organization_id) return json({ error: 'organization_id é obrigatório.' }, 400);
// organization_id é usado como filtro, nunca validado contra o usuário
...
```

Verificado assim: Três arquivos lidos integralmente; nenhuma consulta a `organization_members` em nenhum deles.

`supabase/functions/asaas-charge/index.ts` (linhas 84-115):

...

```
const { data: ch } = await admin.from('client_charges')
  .select('asaas_payment_id,billing_type,party_email,status')
  .eq('id', chargeId).eq('organization_id', organization_id).maybeSingle();
```

```
const emailOverride = body.email ?? ch.party_email;
```

```
if (emailOverride) sendBody.emails = [emailOverride];
await fetch(`${asaasBase}/payments/${ch.asaas_payment_id}/sendByMail`, { ... });
...
```

Verificado assim: Arquivo lido integralmente; caminho de 'resend' nas linhas 84-115.

Impacto

- Operações financeiras reais (emissão, cancelamento, pagamento de boleto) executadas sobre dados de outro tenant.
- Exfiltração de boletos e dados de cobrança de outro tenant para endereço arbitrário.

Correção sugerida

****C2-03:**** Nas três funções, logo após getUser(), consultar organization_members filtrando por organization_id e pelo e-mail/uid do usuário, devolvendo 403 se não houver linha – o mesmo bloco que invite-member/index.ts:51-60 já implementa. Extrair para um módulo compartilhado em supabase/functions/_shared/ e reusar em toda função que receba organization_id.

****C3-05:**** Corrigir C2-03 (checagem de associação) e, adicionalmente, restringir emailOverride aos endereços já cadastrados daquele cliente, em vez de aceitar qualquer string.

Critérios de aceite

- [] Chamada a asaas-charge com organization_id de outra org retorna 403
- [] Idem para asaas-payment e sign-contract
- [] Existe um helper compartilhado de checagem de associação usado pelas três
- [] Os fluxos legítimos de cobrança, pagamento e assinatura seguem funcionando
- [] resend com charge_id de outra organização retorna 403
- [] email fora dos endereços cadastrados do cliente é rejeitado
- [] Segunda via legítima continua sendo enviada
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

[Segurança] sign-contract altera e lê documentos de assinatura sem checagem de posse

****Labels:**** `security`, `severidade:alta`, `multi-tenant`

Problema

Depois de validar que existe um usuário, a função escreve com adminClient (service_role) em quatro tabelas usando ids que vieram direto do corpo: documentVersionId, addendumId, contractId e dealId. Nenhum é conferido contra a organização do chamador – o organizationId até é exigido na linha 60, mas nunca é usado para validar coisa alguma. Um usuário autenticado de qualquer tenant sobrescreve signature_token, signature_status e signature_url de contratos alheios. Na action 'status' o problema é de leitura: signatureToken vem do corpo e a função consulta a API do ZapSign com o token da conta corporativa, devolvendo signers e a URL do arquivo assinado de qualquer documento daquela conta – vazamento de contrato assinado de outro cliente. A action 'webhook' (linhas 166-225) fecha o conjunto: qualquer autenticado marca um contrato como SIGNED informando o token.

Evidência

`supabase/functions/sign-contract/index.ts` (linhas 107-163):

```
...
if (target === 'document_version') {
  await adminClient.from('contract_document_versions').update({
    signature_token: zapDoc.token, signature_status: 'SENT', signature_url: signUrl,
  }).eq('id', documentVersionId); // id veio do body, sem checagem de posse
...
if (action === 'status') {
  const { signatureToken } = body;
  const zapResp = await fetch(`${ZAPSIGN_API}/docs/${signatureToken}/`, { ... });
  return json({ status: zapDoc.status, signers: zapDoc.signers, signed_file: zapDoc.signed_file });
...

```

Verificado assim: Arquivo lido integralmente; organizationId exigido na linha 60 e não referenciado em nenhuma consulta.

Impacto

- Adulteração do estado de assinatura de contratos de outros tenants e leitura de documentos assinados alheios via ZapSign.

Correção sugerida

Validar associação do usuário a organizationId; carregar a linha alvo e conferir que seu organization_id bate com o do chamador antes de qualquer update; na action 'status', localizar primeiro a linha local que possui aquele signature_token dentro da organização do usuário e só então consultar o ZapSign. A action 'webhook' deve sair desta função e virar endpoint próprio, autenticado por segredo compartilhado com o ZapSign.

Critérios de aceite

- [] Enviar para assinatura um contractId de outra organização retorna 403
- [] action 'status' com signatureToken fora da organização retorna 403
- [] A rota de webhook não aceita mais ser chamada por JWT de usuário
- [] Envio, consulta e retorno de assinatura seguem funcionando no fluxo legítimo
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

[Segurança] sinapi-import permite a qualquer usuário logado reescrever a base de preços global

Labels: `security`, `severidade:alta`, `multi-tenant`

Problema

A função verifica apenas que existe um usuário válido – nunca qual o papel dele nem a que organização pertence – e depois grava com adminClient (service_role), que ignora a RLS. A RLS de sinapi_items foi escrita justamente para impedir isso: leitura para authenticated/anon, escrita somente para service_role. A Edge Function contorna essa proteção em nome de quem chamar. sinapi_items e sinapi_references são dados GLOBAIS, sem organization_id: são a tabela de preços usada por todos os orçamentos de todos os tenants. Do lado do frontend não há sequer gate de papel – SinapiImportModal é aberto por DatabaseExplorer.tsx:1621 sem nenhuma checagem de isAdmin. O upsert é por (code, reference_date), então o atacante sobrescreve preços existentes, não apenas acrescenta.

Evidência

`supabase/functions/sinapi-import/index.ts` (linhas 47-89):

```
...
```

```
const { data: { user }, error: authError } = await userClient.auth.getUser();
if (authError || !user) return json({ error: 'Unauthorized' }, 401);
```

```
const adminClient = createClient(supabaseUrl, serviceKey, { ... });
// ... nenhuma checagem de papel entre as duas coisas ...
const { error } = await adminClient
  .from('sinapi_items')
  .upsert(items, { onConflict: 'code,reference_date', ignoreDuplicates: false });
...
```

Verificado assim: Código lido integralmente; RLS de sinapi_items/sinapi_references confirmada em pg_policies (escrita só service_role).

Impacto

- Corrupção da base de preços que alimenta os orçamentos de todos os clientes do SaaS; orçamentos passam a sair com valores manipulados.

Correção sugerida

Exigir papel privilegiado no servidor antes do upsert: consultar organization_members pelo e-mail do usuário validado e aceitar somente owner/admin (ou um papel de superadmin dedicado, já que o dado é global) – exatamente o padrão que invite-member/index.ts:51-60 já usa. Adicionar também o gate de papel na UI, para coerência.

Critérios de aceite

- [] Chamada a sinapi-import com JWT sem papel privilegiado retorna 403
- [] Chamada com owner/admin continua importando a competência
- [] O botão de importação SINAPI só aparece para papel privilegiado
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 8 ---

--- ISSUE 9 ---

[Segurança] send-bi-report é um relay de e-mail aberto a qualquer usuário autenticado

****Labels:**** `security`, `severidade:alta`, `multi-tenant`

Problema

Depois de confirmar que existe um usuário, a função repassa para a Resend, sem nenhuma validação, três campos inteiramente controlados pelo chamador: a lista de destinatários, o assunto e o corpo HTML. O campo organizationId é recebido e nunca usado em consulta alguma. O remetente é o domínio verificado da empresa (relatorios@opura.com.br, linha 57). Qualquer usuário cadastrado – inclusive alguém que acabou de se cadastrar, já que o self-signup está habilitado – dispara e-mail com HTML arbitrário para qualquer endereço, assinado pelo domínio da empresa. É o vetor clássico de phishing com reputação emprestada, e queima a reputação do domínio no envio em massa. O scheduleId também é usado sem checagem de posse: a linha 85 atualiza bi_report_schedules por id, com service_role.

Evidência

`supabase/functions/send-bi-report/index.ts` (linhas 37-71):

...

```
const { data: { user }, error: authError } = await userClient.auth.getUser();
if (authError || !user) return json({ error: 'Token inválido' }, 401);
```

```
const { recipients, subject, htmlBody, scheduleId, organizationId } = await req.json();
if (!recipients?.length) return json({ error: 'Nenhum destinatário informado.' }, 400);
```

```
const sendRes = await fetch('https://api.resend.com/emails', {
  body: JSON.stringify({ from, to: recipients, subject, html: htmlBody }),
});
...;
```

Verificado assim: Código lido integralmente; nenhuma consulta a organization_members no arquivo.

Impacto

- Phishing e spam saindo do domínio corporativo verificado; atualização de agendamentos de relatório de outros tenants.

Correção sugerida

Validar que o usuário é membro de organizationId; derivar os destinatários no servidor a partir de bi_report_schedules daquela organização em vez de aceitar a lista do body; montar o HTML no servidor a partir dos dados do relatório; e filtrar scheduleId por organization_id.

Critérios de aceite

- [] Chamada com organizationId de outra organização retorna 403
- [] Destinatários fora do agendamento da organização são rejeitados
- [] O corpo do e-mail não vem mais cru do cliente
- [] O envio agendado legítimo continua funcionando
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 9 ---

--- ISSUE 10 ---

[Segurança] asaas-webhook falha aberto quando ASAAS_WEBHOOK_TOKEN não está configurada

****Labels:**** `security`, `severidade:alta`, `multi-tenant`

Problema

A condição começa por “webhookToken &&”. Se ASAAS_WEBHOOK_TOKEN não estiver definida no ambiente do projeto – ou for string vazia, ou for removida numa troca de ambiente – a expressão curto-circuita e a validação inteira é pulada: a função passa a aceitar qualquer POST anônimo. Daí em diante o corpo é processado com service_role: PAYMENT_RECEIVED marca client_charges como paga, faz a baixa do recebível em internal_transactions (business_status RECEBIDO, status CONCILIATED) e insere lançamento de taxa; BILL_PAID marca supplier_payments e boletos como pagos. O identificador precisa apenas casar com um asaas_payment_id/asaas_bill_id existente. É uma falha fail-open: o modo inseguro é o default silencioso. A linha 33 ainda registra em log um prefixo e um sufixo do token esperado, junto do comprimento – vazamento parcial de segredo no log.

Evidência

`supabase/functions/asaas-webhook/index.ts` (linhas 28-36):

```
...
const webhookToken = Deno.env.get('ASAAS_WEBHOOK_TOKEN') ?? '';
const incoming = req.headers.get('asaas-access-token') ?? '';
console.log(`[asaas-webhook] incoming=${mask(incoming)} expected=${mask(webhookToken)} ...`);
if (webhookToken && incoming !== webhookToken) {
  return json({ error: 'Invalid webhook token' }, 401);
}
...
```

Verificado assim: Arquivo lido integralmente; condição de guarda na linha 34.

Impacto

- Baixa fraudulenta de contas a receber e a pagar por requisição anônima, caso a variável não esteja configurada.
- > ****Condição de explorabilidade (C3-04):**** Explorabilidade condicionada a ASAAS_WEBHOOK_TOKEN ausente ou vazia no ambiente do projeto Supabase.

Correção sugerida

Inverter para fail-closed: se webhookToken estiver vazio, responder 503 e não processar nada. Comparar em tempo constante. Remover o log das linhas 32-33 ou reduzi-lo a um booleano de match. Adicionar validação de inicialização que recuse subir sem a variável.

Critérios de aceite

- [] Com ASAAS_WEBHOOK_TOKEN ausente, a função responde 503 e não altera nenhuma linha
- [] POST com token errado responde 401
- [] O log não contém mais nenhum trecho do token esperado
- [] O webhook real do Asaas continua sendo processado
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 10 ---

--- ISSUE 11 ---

[Segurança] Ausência de sanitização de HTML nos quatro sinks de innerHTML do frontend

Labels: `security`, `severidade:alta`, `multi-tenant`

Problema

C5-01 - XSS armazenado na Academia: qualquer membro injeta HTML servido a todos os matriculados

conteudo_html vem da tabela academy_lessons e é injetado sem qualquer sanitização. As policies de escrita dessa tabela são academy_lessons_insert e academy_lessons_update, ambas com expressão is_org_member(org_id) – ou seja, QUALQUER membro da organização, no papel mais baixo, escreve o HTML. O conteúdo é depois renderizado para todo colaborador matriculado que abrir a aula, incluindo owners e admins. Não existe biblioteca de sanitização no projeto: package.json não declara DOMPurify nem equivalente, e não há utilitário próprio de escape. Um membro comum publica aula com e executa script na sessão de um administrador – o token do Supabase é acessível ao JavaScript da página, então o resultado é sequestro de sessão privilegiada. É escalada de privilégio dentro do tenant.

C5-02 - Template de contrato injetado via innerHTML na geração de PDF

body_html vem de contract_templates, cuja policy de escrita é contract_templates_org com is_org_member(organization_id) – de novo, qualquer membro da organização. O HTML é atribuído via innerHTML a um elemento que é efetivamente anexado ao documento (linha 893), no contexto de quem gera o PDF, tipicamente um usuário do financeiro ou administrador. innerHTML não executa <script>, mas executa handlers de erro e de carga: e <svg onload=...> disparam normalmente. O container fica fora da tela, então a vítima não vê nada acontecer. Não há sanitização em renderTemplate nem no ponto de uso.

C5-04 - Preview de template renderiza HTML do próprio autor sem sanitização (self-XSS)

Nos dois casos o HTML renderizado é o que o próprio usuário acabou de digitar no textarea ao lado (previewContent vem de renderTemplate(bodyHtml, ...) na linha 191; previewBody vem de form.body_template na linha 101 do DunningModule). Sozinho isso é self-XSS: a vítima teria de colar o payload em si mesma, o que não configura vulnerabilidade explorada por terceiro. Fica registrado como baixa por dois motivos: é o mesmo dado que, depois de salvo, alimenta o sink de C5-02 e o corpo dos e-mails de cobrança; e a correção é a mesma linha de código, então separar as duas passadas só cria retrabalho.

Evidência

`components/academy/AcademyLessonPlayer.tsx` (linhas 180-183):

```
...
case 'TEXT0':
  return (
    <div
      className="prose prose-sm max-w-none ..."
      dangerouslySetInnerHTML={{ __html: lesson.conteudo_html || '' }}
    />
  );
...
```

Verificado assim: Sink lido no arquivo; policies de academy_lessons confirmadas em pg_policies (is_org_member); ausência de lib de sanitização confirmada em package.json.

`components/ContractDetailView.tsx` (linhas 885-893):

```
...
const rendered = renderTemplate(template.body_html, varMap);

const container = window.document.createElement('div');
container.style.cssText = 'position:fixed;left:-9999px;top:0;width:794px;...';
container.innerHTML = rendered;
window.document.body.appendChild(container);
...
```

Verificado assim: Sink lido no arquivo; policy contract_templates_org confirmada em pg_policies.

`components/ContractTemplateManager.tsx` (linhas 248-252 (e components/DunningModule.tsx:243-246)):

```

...
{previewMode ? (
  <div
    className="min-h-[480px] ... prose prose-sm max-w-none"
    dangerouslySetInnerHTML={{ __html: previewContent }}
  />
) : ( ... )}
...

```

Verificado assim: Origem de previewContent e previewBody rastreada até o estado local do formulário.

Impacto

- Execução de script na sessão de administradores; roubo de sessão e escalada de privilégio dentro da organização.
- Execução de script na sessão de quem gera o PDF do contrato, disparada por qualquer membro da organização.
- Sem impacto direto isolado; relevante como parte da mesma superfície de C5-02.

Correção sugerida

****C5-01:**** Adicionar dependência de sanitização (DOMPurify) e criar um helper único – por exemplo `utils/sanitizeHtml.ts` – com allowlist de tags e atributos, proibindo event handlers e javascript: em href/src. Aplicar em todo `dangerouslySetInnerHTML` que renderize dado de banco. Sanitizar também na escrita, para não depender só da leitura.

****C5-02:**** Sanitizar rendered com o mesmo helper de C5-01 antes da atribuição; aplicar também nos dois pontos de preview de template (C5-04).

****C5-04:**** Aplicar o helper de sanitização de C5-01 nos dois pontos de preview.

Critérios de aceite

- [] DOMPurify (ou equivalente) consta em package.json
- [] Existe helper único de sanitização e `AcademyLessonPlayer` o usa
- [] Teste cobrindo `<img onerror>`, `<script>` e `href=javascript:` prova que são neutralizados
- [] Aula com HTML legítimo (negrito, lista, link http) continua renderizando
- [] `ContractDetailView` sanitiza antes de atribuir a `innerHTML`
- [] `ContractTemplateManager.tsx:251` e `DunningModule.tsx:245` usam o mesmo helper
- [] Teste com `<img onerror>` em `body_html` prova que o handler não dispara
- [] O PDF gerado a partir de template legítimo permanece idêntico
- [] Os dois previews passam pelo helper de sanitização
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 11 ---

--- ISSUE 12 ---

[Segurança] Funções `notify-*` sem autenticação e com HTML de e-mail montado por interpolação

****Labels:**** `security`, `severidade:media`

Problema

****C3-06 - notify-broker-proposal e notify-opportunity-interest não verificam Authorization nenhuma****

Ao contrário das demais, estas duas funções não leem o header `Authorization` em momento algum. Elas filtram corretamente por `organizationId` nas consultas (o que impede montar notificação com dados de outro tenant – ponto positivo), mas qualquer um que conheça um par válido de ids dispara notificação in-app e e-mail para todos os owners/admins daquela organização, quantas vezes quiser. Serve como amplificador de spam e de phishing interno, já que o conteúdo do e-mail vem de campos gravados por formulário público (ver C5-03). Ainda que o `verify_jwt` da plataforma esteja ligado, ele é satisfeito pela chave anon, que é pública.

****C5-03 - E-mails HTML montados por interpolação, com campo vindo de formulário público anônimo****

O HTML do e-mail é montado por template string, sem nenhum escape. Os campos `contact_name`, `contact_email`, `contact_phone` e `message` vêm de `opportunity_interests`, alimentada pelo formulário público de manifestação de interesse (RPC `fn_investor_portal_submit_interest`, confirmada como executável por anon) – ou seja, são texto de atacante anônimo. Em `notify-broker-proposal` o mesmo vale para `notes`, `buyer_name` e `broker_email`. O destinatário é a caixa de todos os owners/admins da organização. Clientes de e-mail modernos bloqueiam script, então não é XSS clássico; o que se

consegue é quebrar a estrutura do HTML e injetar conteúdo e âncoras arbitrárias — um `<a href>` convincente dentro de uma mensagem que chega do domínio corporativo da própria empresa. Combinado com C3-06 (as duas funções não exigem credencial), o atacante escolhe a hora e a frequência do disparo. `contact_email` ainda vai para dentro de um atributo `href`, o contexto mais frouxo dos presentes.

Evidência

`supabase/functions/notify-broker-proposal/index.ts` (linhas 22-40 (e `notify-opportunity-interest/index.ts:28-53`)):

```
...
serve(async (req: Request) => {
  if (req.method === 'OPTIONS') return new Response('ok', { headers: corsHeaders });

  const supabaseUrl = Deno.env.get('SUPABASE_URL') ?? '';
  const serviceRoleKey = Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') ?? '';
  // nenhuma leitura de req.headers.get('Authorization') em todo o arquivo
  const { proposalId, organizationId } = await req.json();
...

```

Verificado assim: Dois arquivos lidos integralmente; ausência de qualquer leitura de `Authorization`.

`supabase/functions/notify-opportunity-interest/index.ts` (linhas 134-174 (e `notify-broker-proposal/index.ts:128-149`)):

```
...
<p style="...">${opp.title}</p>
${opp.subtitle} ? `<p style="...">${opp.subtitle}</p>` : ''
<span style="...">${interest.contact_name}</span>
<a href="mailto:${interest.contact_email}" style="...">${interest.contact_email}</a>
<p style="...">${interest.message}</p>
...

```

Verificado assim: Duas funções lidas integralmente; `fn_investor_portal_submit_interest` confirmada como executável por `anon` em `pg_proc`.

Impacto

- Spam ilimitado de notificação e e-mail para administradores, com conteúdo parcialmente controlado pelo atacante.
- Phishing dirigido a administradores, com conteúdo controlado pelo atacante, entregue pelo domínio corporativo.

Correção sugerida

****C3-06:**** Adotar o gate já usado nas funções de cron (comparar `Authorization` com `Bearer <service_role>`) e chamar estas funções a partir de `trigger/serviço`, ou exigir sessão válida com associação a `organizationId`. Somar `rate limiting` por (`organizationId`, `interestId`).

****C5-03:**** Criar helper de escape de HTML (e um específico para contexto de atributo) em `supabase/functions/_shared/` e aplicar a toda interpolação de dado do banco nas duas funções. Validar `contact_email` como e-mail antes de usá-lo em `href`, e truncar `message` a um tamanho máximo.

Critérios de aceite

- [] Chamada sem credencial válida retorna 401
- [] O fluxo real de nova proposta / novo interesse continua notificando os admins
- [] Existe helper de escape compartilhado e as duas funções o usam em toda interpolação
- [] Interesse com `<a href>` ou aspas no campo `message` chega ao e-mail como texto literal
- [] `contact_email` inválido não é renderizado como `href`
- [] O e-mail legítimo continua com o mesmo layout
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 12 ---

--- ISSUE 13 ---

```
## [Segurança] partner-portal-upload aceita contractId arbitrário e content-type do usuário em bucket público

**Labels:** `security`, `severidade:media`

### Problema

O ramo target='invoice' nunca confere que contractId pertence ao workspace do token validado – diferente do ramo padrão, que usa o workspaceId derivado do token no caminho. Um parceiro qualquer grava arquivo na pasta de qualquer contrato. O bucket documents é público (confirmado em storage.buckets: public=true, allowed_mime_types NULL, file_size_limit NULL) e a função aceita o content-type informado pelo cliente, então dá para publicar text/html arbitrário numa URL do domínio de storage da organização – útil para phishing e para hospedar carga maliciosa com ar de legitimidade. Não há limite de tipo nem de tamanho no bucket.

### Evidência

`supabase/functions/partner-portal-upload/index.ts` (linhas 88-102):
...
if (target === 'invoice') {
  if (!contractId) return json({ error: 'contractId é obrigatório ...' }, 400);
  const path = `invoices/${contractId}/${Date.now()}_${safeName}`;
  const { error: uploadError } = await admin.storage.from('documents')
    .upload(path, bytes, { contentType: file.type || 'application/octet-stream', ... });
  const { data: pub } = admin.storage.from('documents').getPublicUrl(path);
  return json({ publicUrl: pub.publicUrl });
}
...

_Verificado assim:_ Arquivo lido integralmente; flag public=true confirmada em storage.buckets no banco remoto.

### Impacto

- Contaminação de anexos de contratos alheios e hospedagem de conteúdo arbitrário (inclusive HTML) em URL pública da organização.

### Correção sugerida

Validar que contractId pertence ao workspace do token antes de montar o caminho; forçar allowlist de content-type (application/pdf e imagens) em vez de confiar em file.type; definir allowed_mime_types e file_size_limit no bucket documents; avaliar torná-lo privado com URL assinada, no padrão já adotado para opura-docs.

### Critérios de aceite

- [ ] Upload com contractId de outro workspace retorna 403
- [ ] Upload de text/html é recusado
- [ ] O bucket documents tem allowed_mime_types e file_size_limit definidos
- [ ] Envio de NF pelo Portal do Parceiro continua funcionando
- [ ] Existe teste automatizado que falha se a condição voltar
```

--- FIM ISSUE 13 ---

--- ISSUE 14 ---

```
## [Segurança] Policies permissivas remanescentes em opura_cno_* e payment_types

**Labels:** `security`, `severidade:media`

### Problema

**C1-03 - Policies anon FOR ALL USING(true) sobrevivendo em opura_cno_areas e opura_cno_reductions**

São duas das policies “Regra 8 / Dev” que a migration 20270208000002 se propôs a eliminar (81 removidas) e que escaparam do rollout. FOR ALL com USING(true) para o papel anon significa leitura, escrita e exclusão por qualquer portador da chave pública. A exploração hoje é limitada porque as duas tabelas estão vazias (verificado por PostgREST: Content-Range */0), mas nada impede a escrita – o anônimo pode popular a tabela – e a exposição passa a ser total no dia em que o módulo CNO receber
```

dados de produção. É um buraco latente, não um buraco fechado.

****C1-04 - payment_types compartilhado entre todos os tenants apesar de ter organization_id****

A tabela tem coluna de tenant, mas as duas policies usam USING(true) para authenticated. Foi a única tabela com coluna de tenant nessa condição em toda a varredura cruzada entre pg_policies e information_schema.columns. Qualquer usuário logado lê, altera e apaga os tipos de pagamento de qualquer organização. O dado em si é catálogo (baixa sensibilidade), mas a escrita permite sabotagem do cadastro alheio e os nomes podem revelar estrutura financeira de outro tenant.

Evidência

`pg\_policies (banco remoto)` (linhas Allow anon all on opura_cno_areas / opura_cno_reductions):

...

tablename	polycyname	cmd	roles	using
opura_cno_areas	Allow anon all on opura_cno_areas	ALL	{anon}	true
opura_cno_reductions	Allow anon all on opura_cno_reductions	ALL	{anon}	true

...

Verificado assim: pg_policies no banco remoto; contagem de linhas via PostgREST com chave anon (0 linhas hoje).

`pg\_policies (banco remoto)` (linhas Allow authenticated users to manage / to read payment types):

...

payment_types	Allow authenticated users to manage payment types	ALL	{authenticated}	true
payment_types	Allow authenticated users to read payment types	SELECT	{authenticated}	true

...

Verificado assim: pg_policies cruzado com information_schema.columns (presença de organization_id).

Impacto

- Leitura e escrita anônima sobre dados de CNO/previdência assim que o módulo entrar em uso.
- Leitura e alteração cruzada de catálogo financeiro entre tenants.

Correção sugerida

****C1-03:** DROP das duas policies, no mesmo padrão da migration 20270208000002, deixando só as policies org-scoped de authenticated.**

****C1-04:** Reescrever as duas policies com is_org_member(organization_id), preservando leitura de linhas globais (organization_id IS NULL) se o catálogo tiver seeds do sistema.**

Critérios de aceite

- [] Nenhuma policy com roles={anon} e qual=true resta em opura_cno_areas/opura_cno_reductions
- [] O módulo CNO continua funcionando para usuário autenticado membro da organização
- [] Usuário do tenant A não lê nem altera payment_types do tenant B
- [] Seeds globais (organization_id IS NULL) continuam visíveis a todos
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 14 ---

--- ISSUE 15 ---

[Segurança] Higiene de segredos: credencial cravada, placeholders de service_role e project ref no código

****Labels:** `security`, `severidade:baixa`**

Problema

****C4-01 - Chave publishable e project ref do Supabase embutidos em script versionado****

A chave publishable/anon é desenhada para ser pública – ela já vai no bundle do frontend – então o vazamento em si não é o problema. O que importa é o efeito combinado: o arquivo está versionado (git ls-files confirma) e dá a qualquer leitor do repositório um cliente pronto, apontado para o projeto de produção, sem passo nenhum de configuração. Foi exatamente com essa chave que o achado C1-02 foi confirmado. Os demais scripts da raiz (test_db.js, test_insert.js, unlink_db.js, query_doc.js) fazem o certo: leem de process.env ou do .env. Este é o único com valor cravado. Registra-se que nenhum

segredo real – service_role, JWT, chave de API – foi encontrado em conteúdo versionado.

****C4-02 - Placeholders de segredo como fallback de COALESCE nos agendamentos pg_cron****

O padrão COALESCE(variável, 'PLACEHOLDER') faz o literal virar o segredo efetivo quando a variável não está configurada. Aqui o efeito imediato é de disponibilidade, não de exposição: as funções alvo comparam o header com Bearer <service_role> e recusam o placeholder, então o cron de faturamento e o de alerta de tarefa falham 401 em silêncio – ninguém é avisado de que pararam. O risco de segurança é o inverso do costume: como o valor certo precisa ser colado à mão (o comentário da linha 47 manda literalmente substituir pela service_role key), o próximo passo natural de quem for consertar é cravar a chave real numa migration versionada. Não há validação de inicialização que recuse o placeholder.

****C4-03 - Project ref do Supabase cravado como fallback de Access-Control-Allow-Origin****

Não é credencial e não concede acesso: o project ref é visível em qualquer requisição do app. Fica o registro como higiene – identificador de ambiente de produção cravado no código, que vira armadilha em fork, em homologação ou numa eventual migração de projeto. Como toda produção real define FRONTEND_URL, o fallback nunca deveria ser exercido.

Evidência

`check\_suppliers.js` (linhas 3-4):

```
...  
const supabaseUrl = 'https://oxedkknreghxrgenyjiu.supabase.co';  
const supabaseKey = 'sb_publishable_IgIC72BIXCLNix4ARLo0QA_0UGDrnzW';  
const supabase = createClient(supabaseUrl, supabaseKey);  
...
```

Verificado assim: git ls-files confirma o arquivo versionado; git grep por padrões de segredo em todo o conteúdo versionado não encontrou segredo real.

`supabase/migrations/20260224000002\_setup\_billing\_cron.sql` (linhas 37 (e 20261118000011_task_alert_sent_at.sql:31-35)):

```
...  
'Authorization', 'Bearer ' || (SELECT COALESCE(current_setting('vault.service_role_key', true),  
  'INTERNAL_SECRET_HERE'))  
  
-- 20261118000011_task_alert_sent_at.sql:  
'Authorization', 'Bearer ' || coalesce(  
  current_setting('vault.service_role_key', true),  
  current_setting('app.service_role_key', true),  
  'CONFIGURE_SERVICE_ROLE_KEY')  
...
```

Verificado assim: Migrations lidas na íntegra; padrão correto identificado em 20260514000002_quality_sla_cron.sql:21.

`supabase/functions/process-billing-ruler/index.ts` (linhas 8-11):

```
...  
const corsHeaders = {  
  'Access-Control-Allow-Origin': Deno.env.get('FRONTEND_URL') ??  
  'https://oxedkknreghxrgenyjiu.supabase.co',  
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',  
}  
...
```

Verificado assim: Arquivo lido integralmente.

Impacto

- Reduz a zero o atrito para explorar qualquer falha de RLS no projeto de produção.
- Jobs de cron falhando silenciosamente e convite estrutural a commitar a service_role key numa migration.
- Nenhum impacto direto de segurança; risco de configuração errada silenciosa entre ambientes.

Correção sugerida

****C4-01:**** Trocar os literais por process.env, como nos demais scripts; ou mover check_suppliers.js

para _dev_scripts/ (já no .gitignore) e remover do índice do git.

****C4-02:**** Remover os fallbacks literais e deixar a chamada falhar de forma visível quando a chave não estiver no Vault; ler o segredo apenas de vault.decrypted_secrets, como a migration 20260514000002 (quality-sla-enforcement) já faz corretamente; e acrescentar teste que recuse migration contendo os literais INTERNAL_SECRET_HERE / CONFIGURE_SERVICE_ROLE_KEY / SUA_SERVICE_ROLE_KEY.

****C4-03:**** Falhar explicitamente quando FRONTEND_URL não estiver definida, em vez de cair num literal de produção.

Critérios de aceite

- [] Nenhum literal de URL/chave de projeto em arquivo versionado
- [] git grep por 'sb_publishable_' e por '.supabase.co' em código não retorna credencial cravada
- [] Nenhuma migration contém os literais de placeholder de segredo
- [] Os dois jobs leem a chave do Vault, no padrão da 20260514000002
- [] Existe verificação mecânica que quebra o build se o padrão voltar
- [] Os jobs de faturamento e de alerta de tarefa executam com 200
- [] Nenhum project ref literal em supabase/functions/
- [] Existe teste automatizado que falha se a condição voltar

--- FIM ISSUE 15 ---